

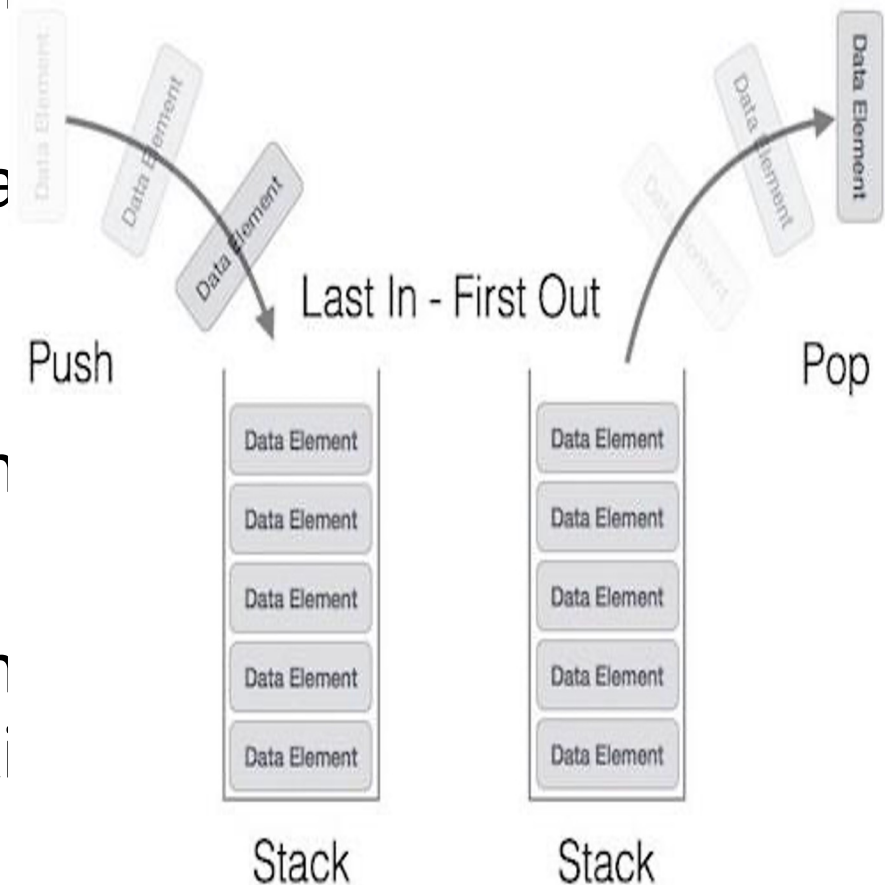
STACK YIĞIT YAPISI

06.03.2024

Hafta 3

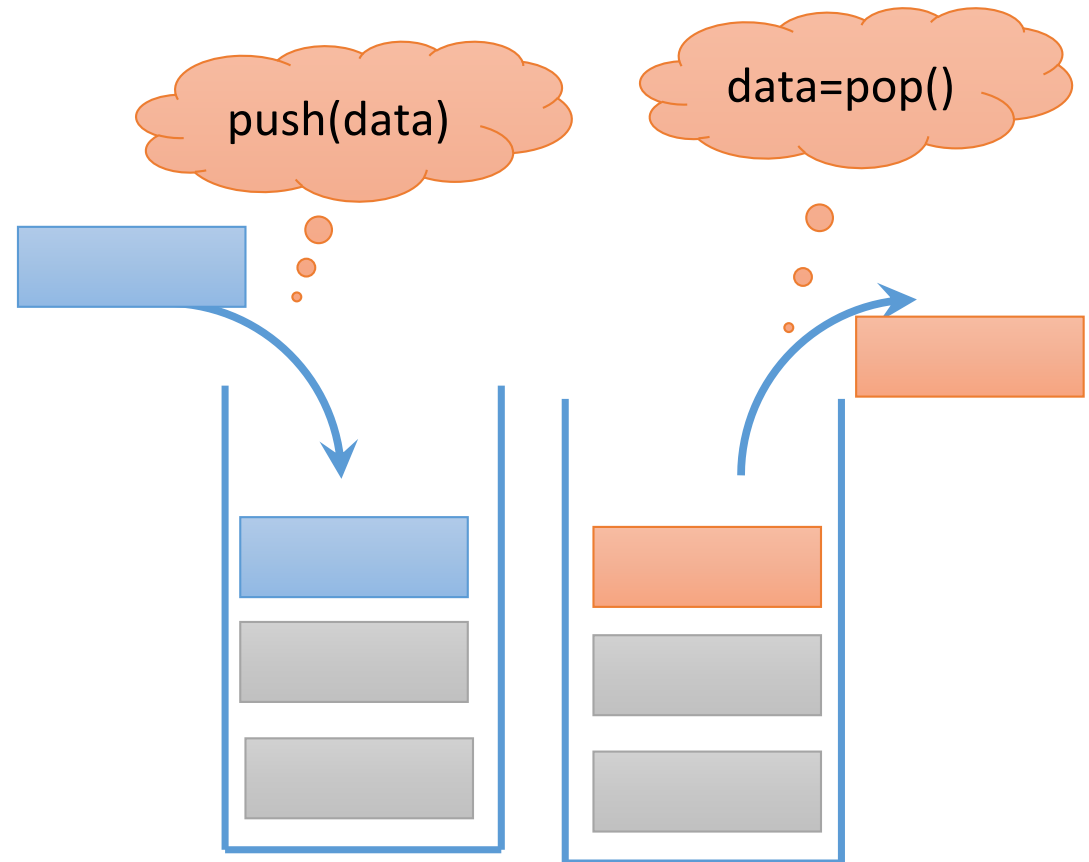
Stack Veri Yapısı

- Stack(Yığıt), elemanları sadece bir uçtan eklenip çıkarılabilen bir veri yapısıdır.
- Son giren ilk çıkar mantığına göre çalışır.
 - Last In First Out (LIFO)
- Stack(Yığıt)' a eklenen son elemanın konumu en üsttedir.
- Stack(Yığıt)' ten bir eleman çıkarılacağı zaman en üstteki eleman çıkartılır.



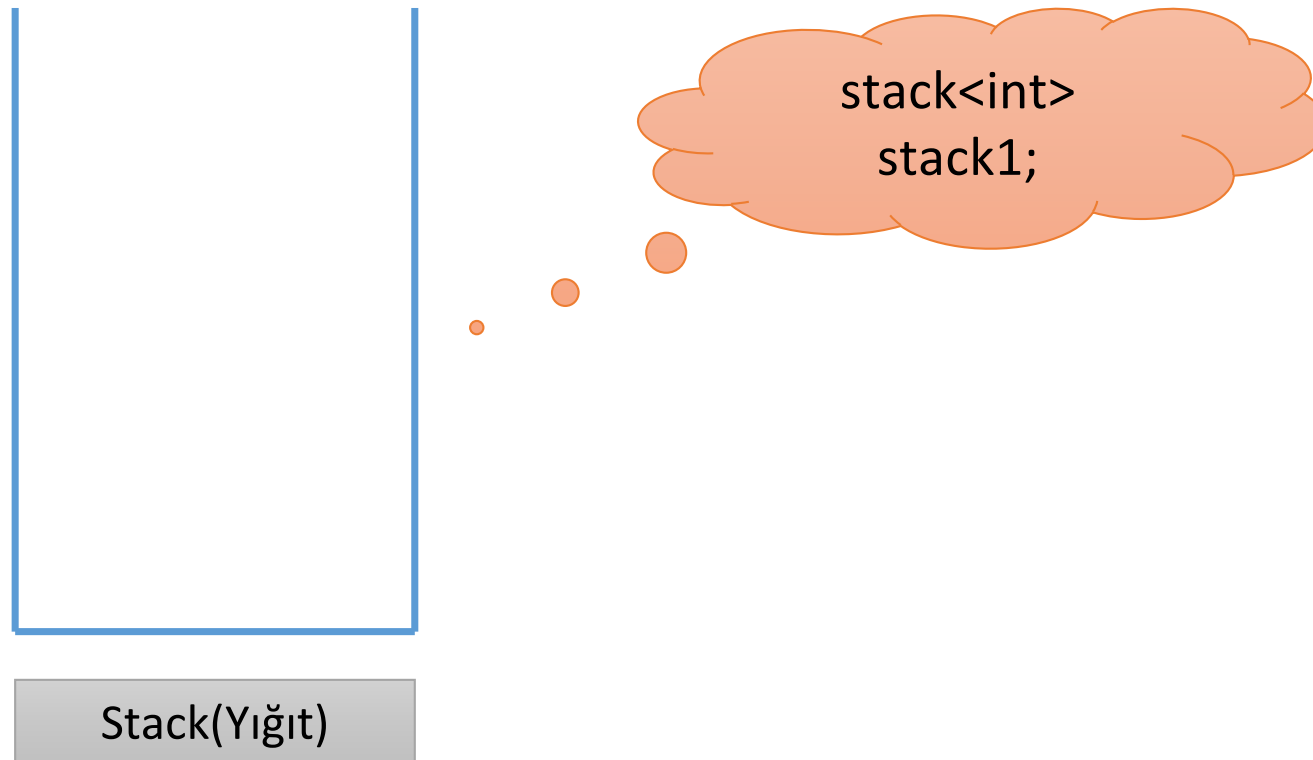
Stack(Yığıt) nasıl çalışır?

- Stack ADT temel işlemleri:
- Stack(Yığıt)' e veri ekleme
 - **void push(data)**
- Stack(Yığıt)' in üstünden veri silme
 - **Data pop()**
- Stack(Yığıt)' in üstünden veri okuma
 - **Data top()** veya **Data peek()**
- Stack(Yığıt)' i boşaltma
 - **void clear()**



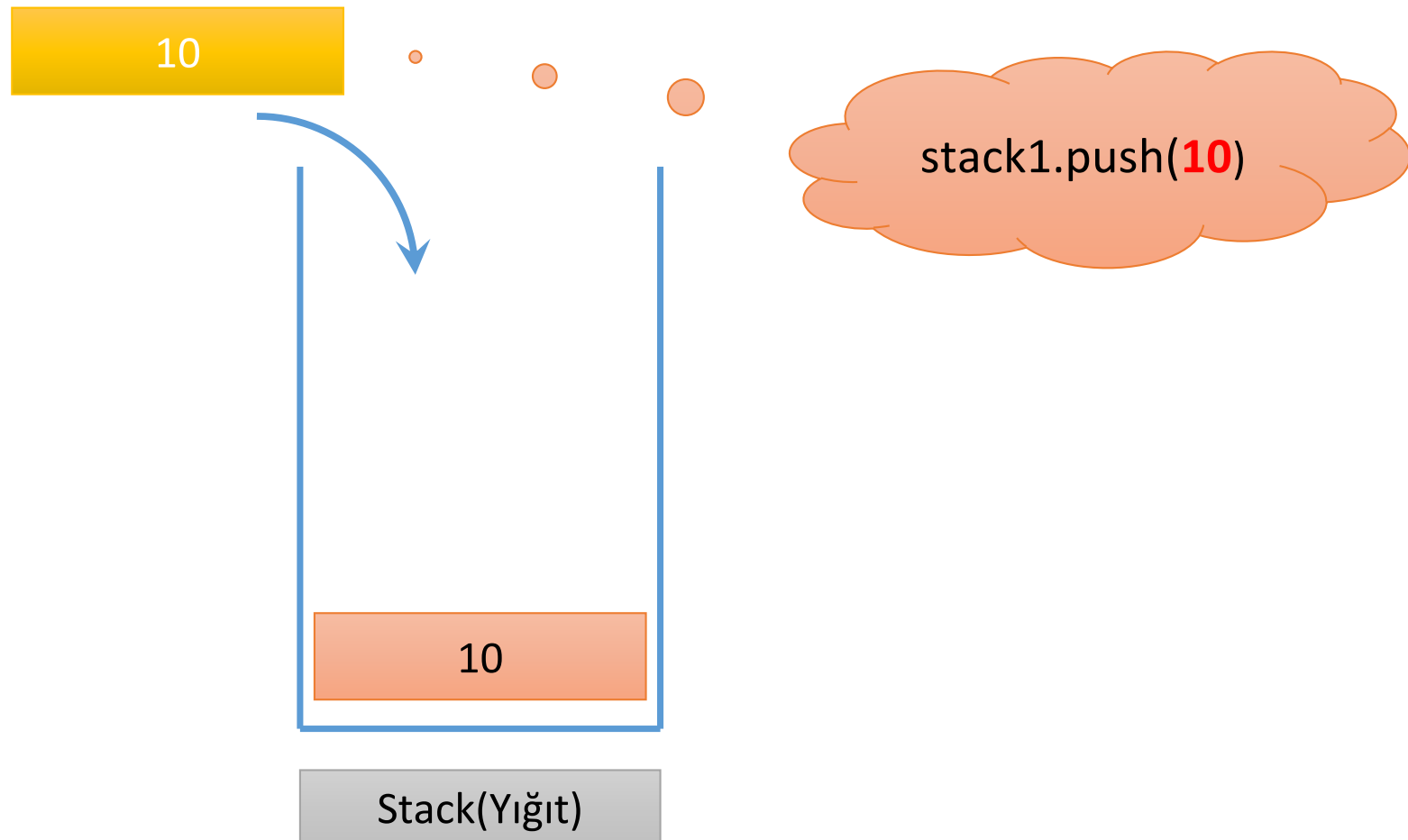
Stack(Yığıt) nasıl çalışır?

- stack sınıfından tanımlanmış stack1 isimli bir Stack(Yığıt)
 - Başlangıçta Stack(Yığıt) boş.



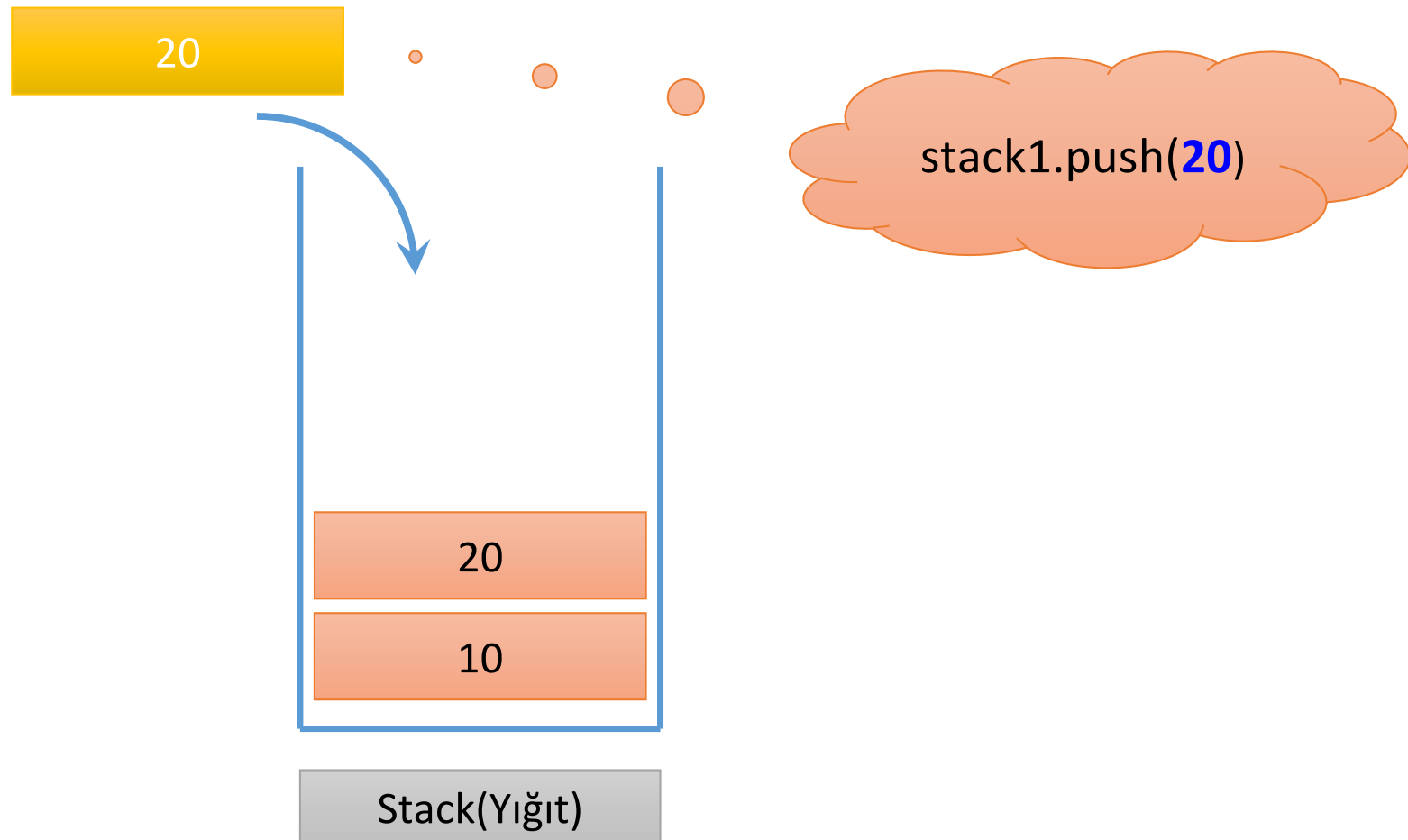
Stack(Yığıt) nasıl çalışır?

- Push ile Stack(Yığıt)ın üstüne yeni bir eleman eklenir.



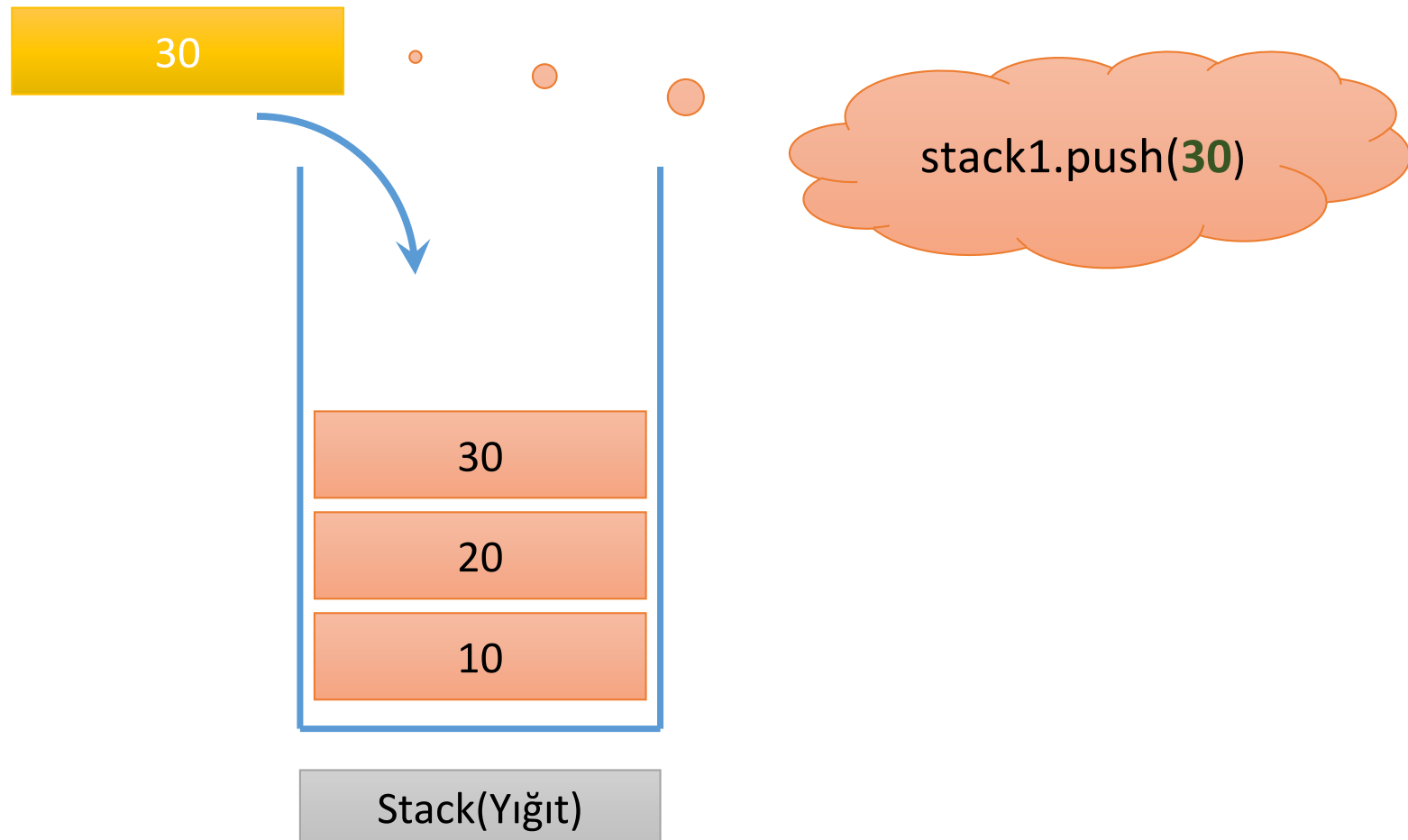
Stack(Yığıt) nasıl çalışır?

- Push ile Stack(Yığıt)a yeni bir eleman eklenir.



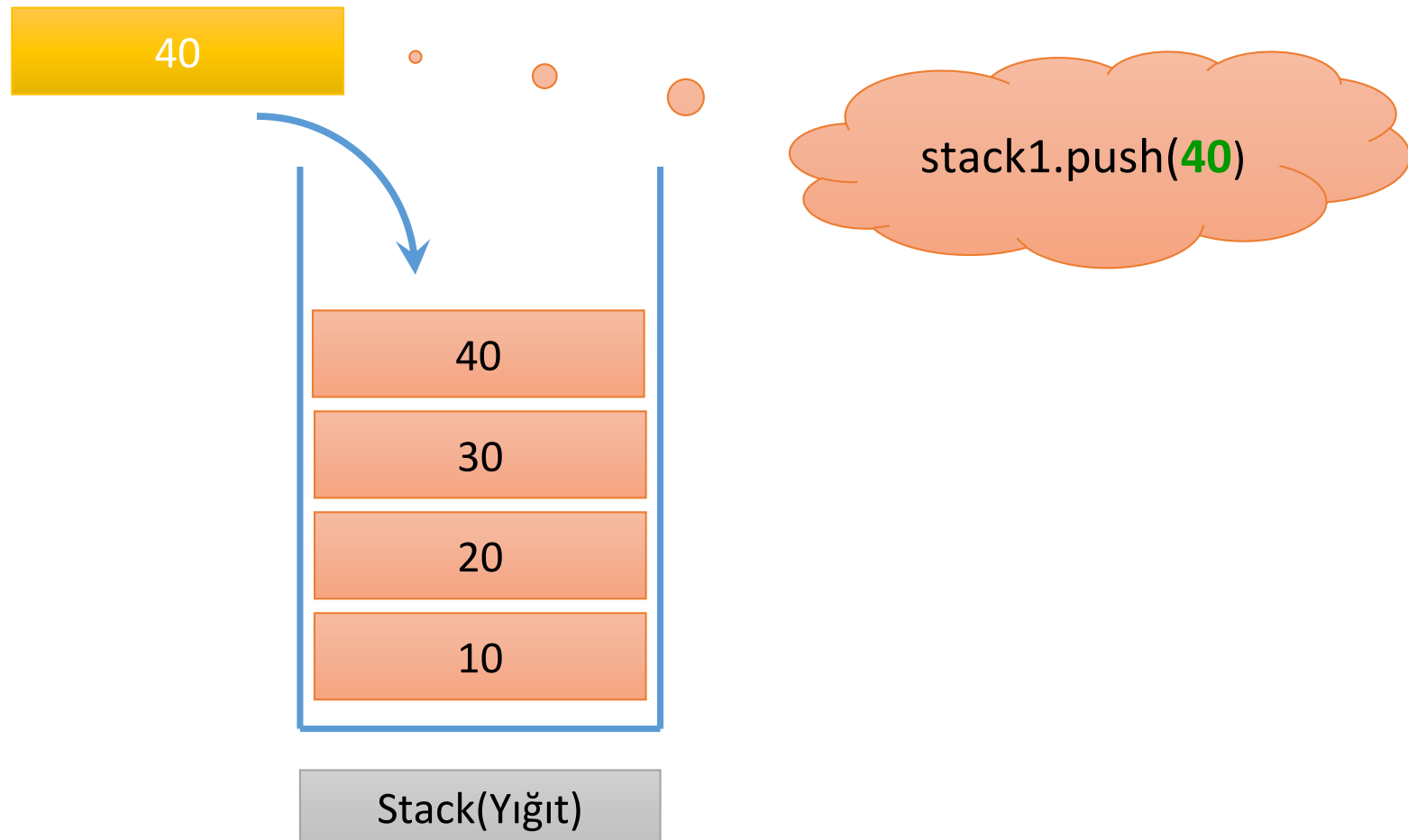
Stack(Yığıt) nasıl çalışır?

- Push ile Stack(Yığıt)a yeni bir eleman eklenir.



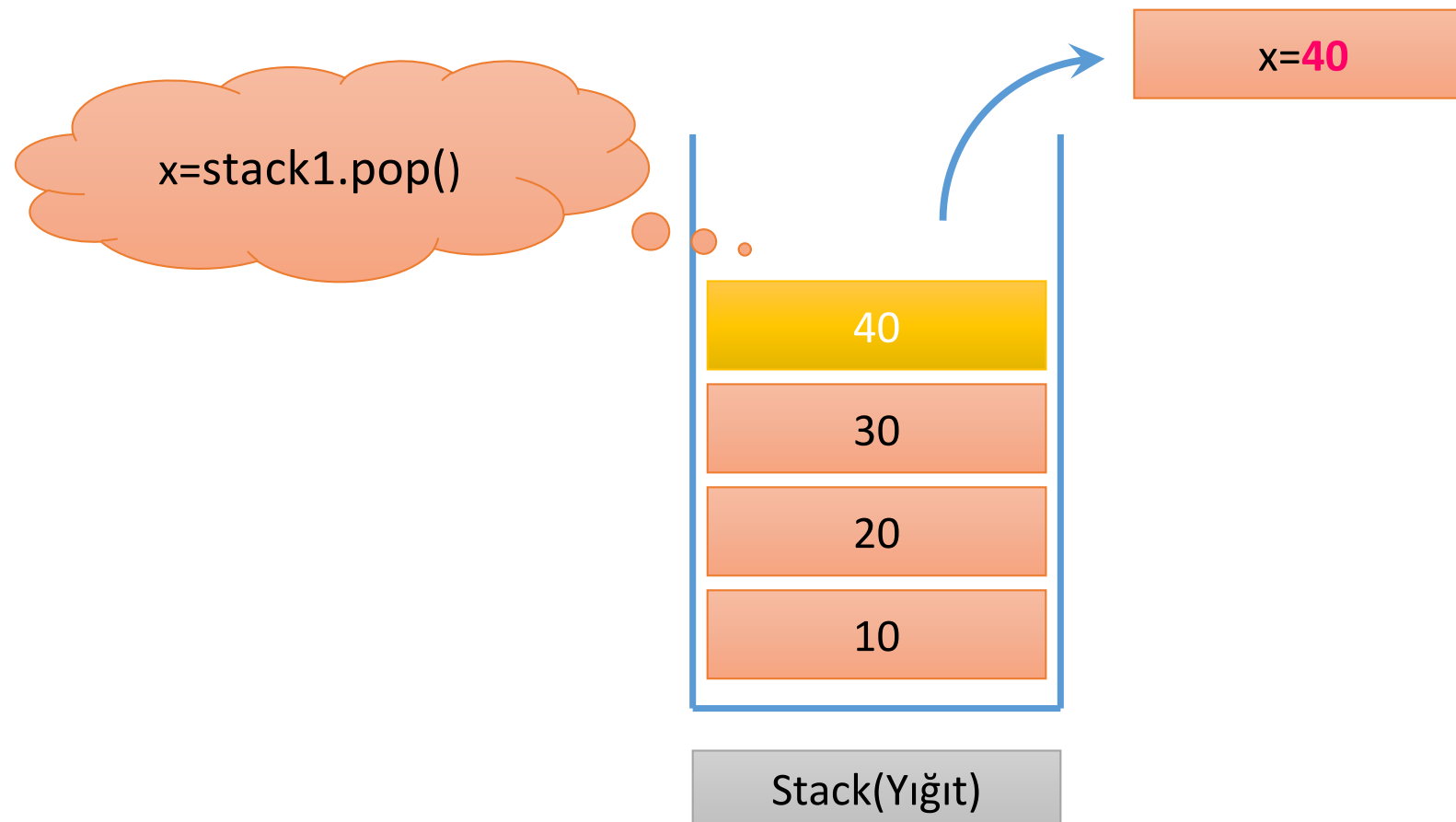
Stack(Yığıt) nasıl çalışır?

- Push ile Stack(Yığıt)ın üstüne yeni bir eleman ekliyoruz.



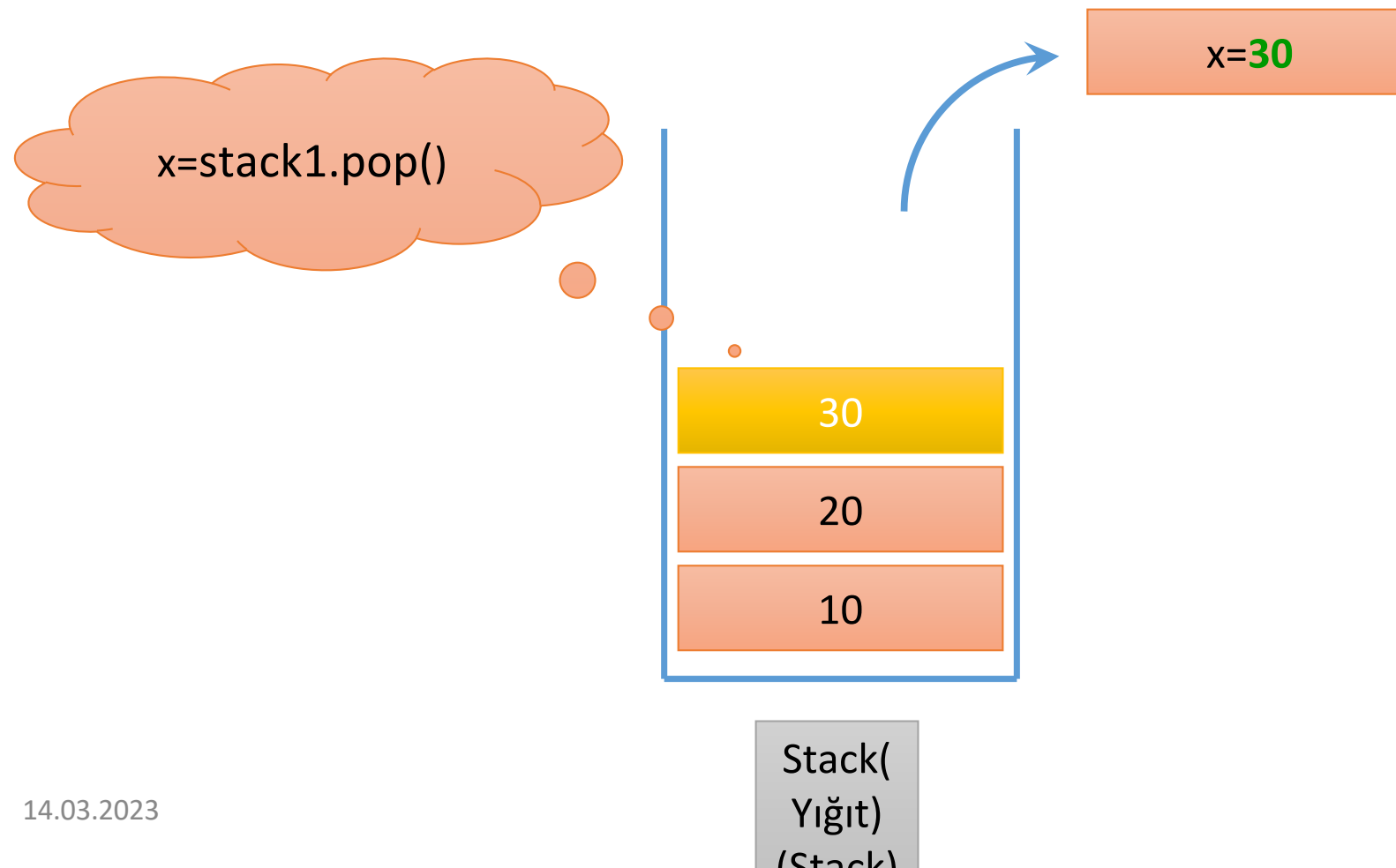
Stack(Yığıt) nasıl çalışır?

- Pop ile Stack(Yığıt)ın en üst elemanı çıkarılır.



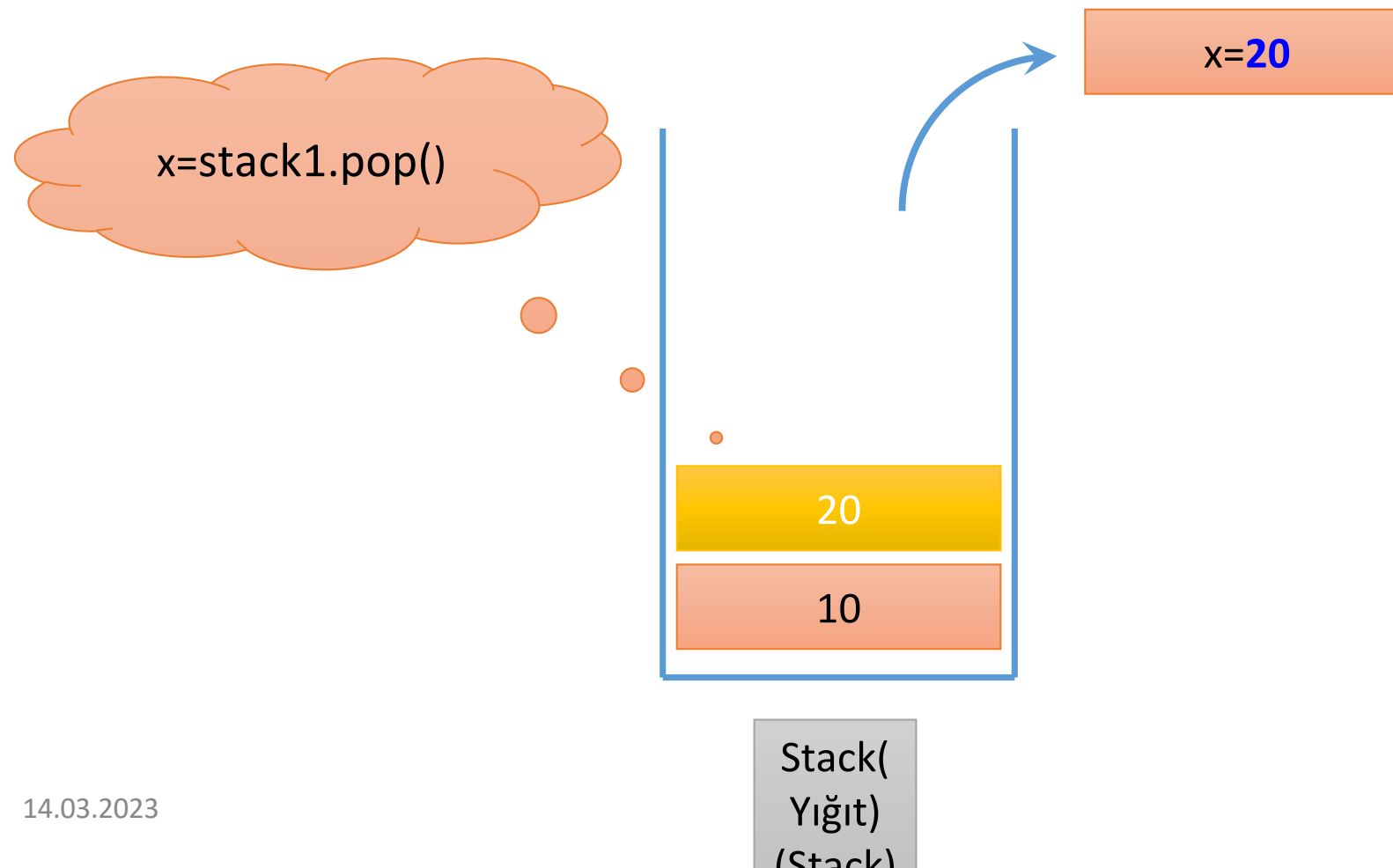
Stack(Yığıt) nasıl çalışır?

- Pop ile Stack(Yığıt)ın en üst elemanı çıkarılır.



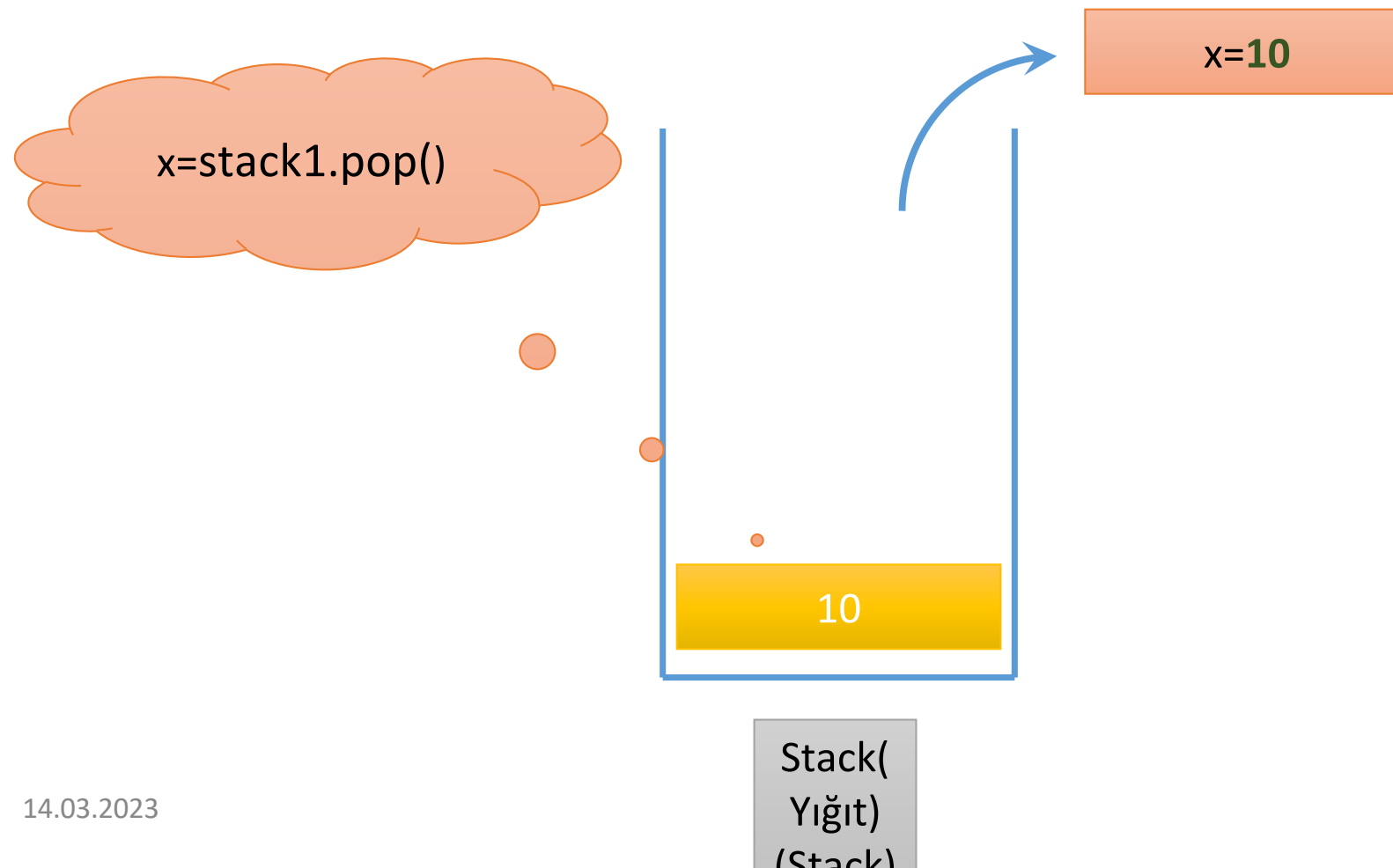
Stack(Yığıt) nasıl çalışır?

- Pop ile Stack(Yığıt)ın en üst elemanı çıkarılır.



Stack(Yığıt) nasıl çalışır?

- Pop ile Stack(Yığıt)ın en üst elemanı çıkarılır.



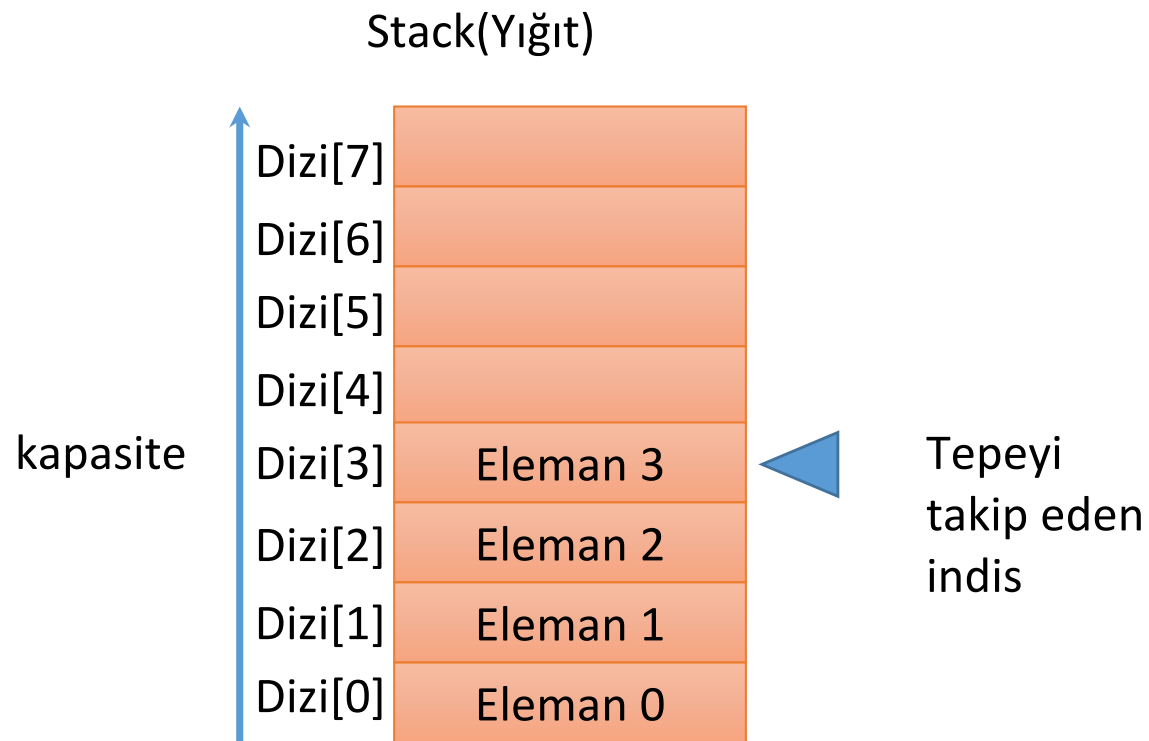
Stack(Yığıt) üzerinde tanımlanabilecek temel işlemler

- **push** : Veriyi Stack(Yığıt)' in üstüne gönder (Ekle).
- **pop** : Stack (Yığıt)' in üstündeki veriyi çıkart.
- **top,peek** : En üstteki veriyi oku. (Stack' e son eklenen veri)
- **isEmpty** : Stack(Yığıt) boş mu?
- **clear** : Stack(Yığıt)' i boşalt.

Stack Performansı?

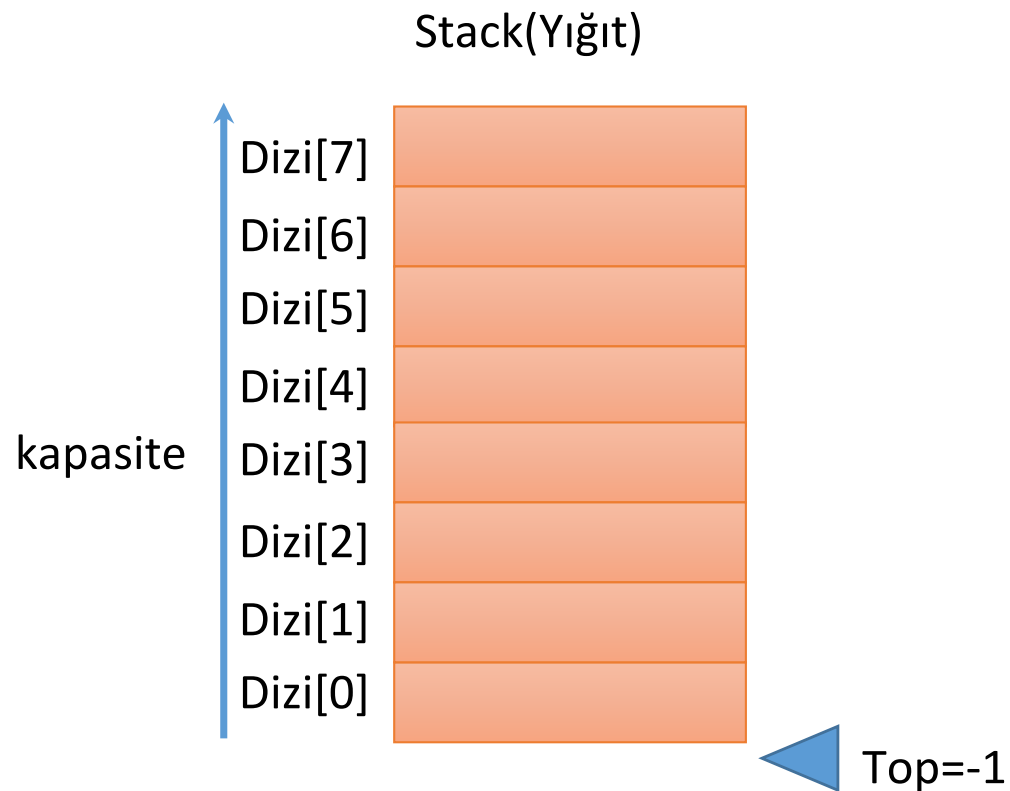
- Push, pop ve peek işlemleri $O(1)$ olmalı?
- Sadece en son veriye erişilmek istenildiği durumlarda kullanılacak bir veri yapısı?
 - Nasıl oluşturulabilir?

Stack – Dizi ile gerçekleştirme

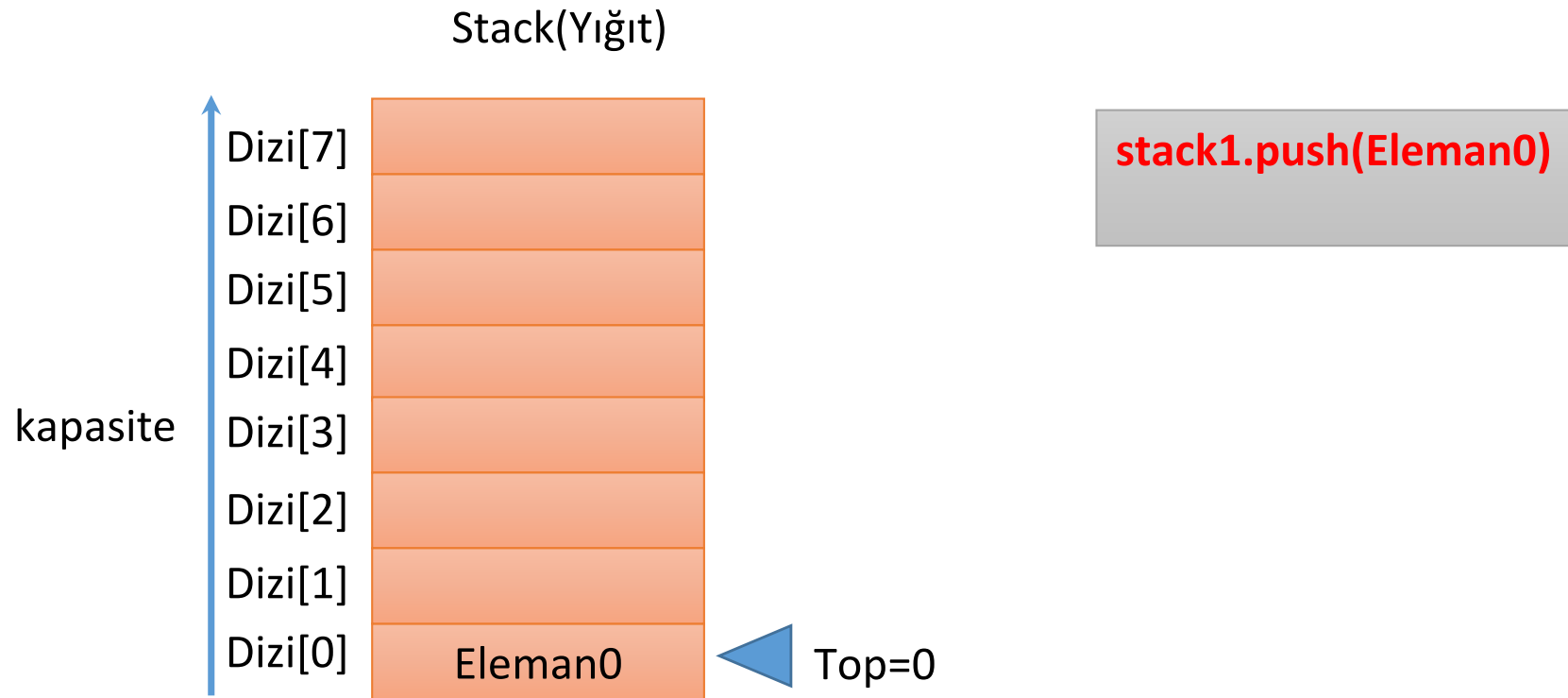


- Dizi ile stack uygulamasında stack veriler dizi üzerinde tutulur.
- Bir tam sayı değişken `Stack(Yığıt)`' in tepesindeki elemanın adresini takip etmek için kullanılır.

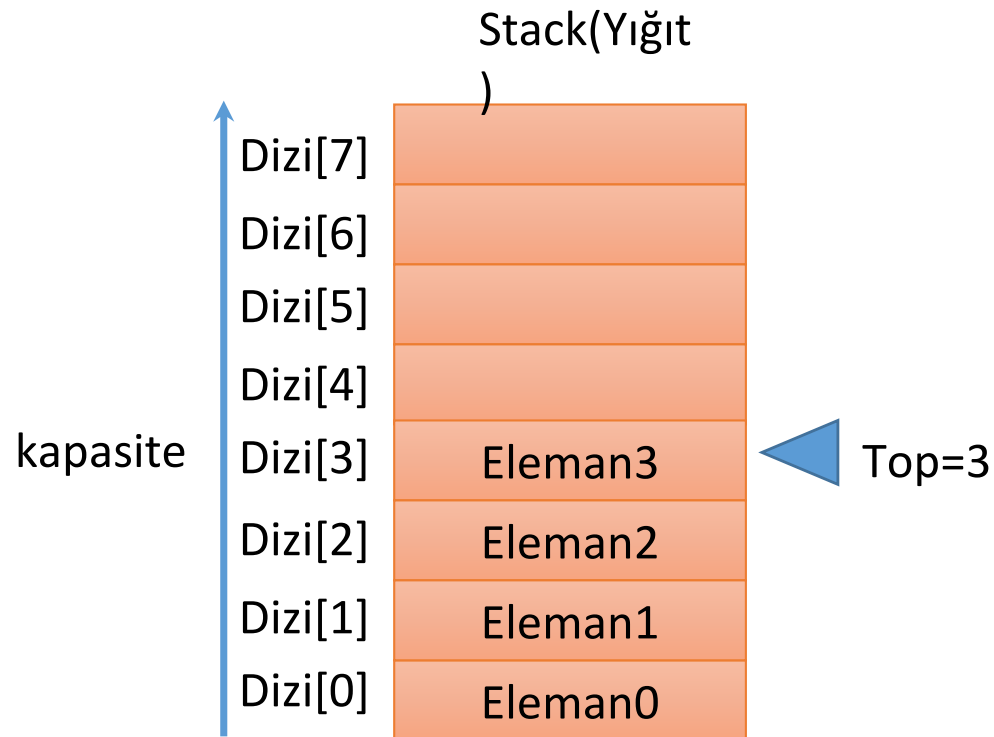
Stack – Dizi ile gerçekleştirme



Stack – Dizi ile gerçekleştirme

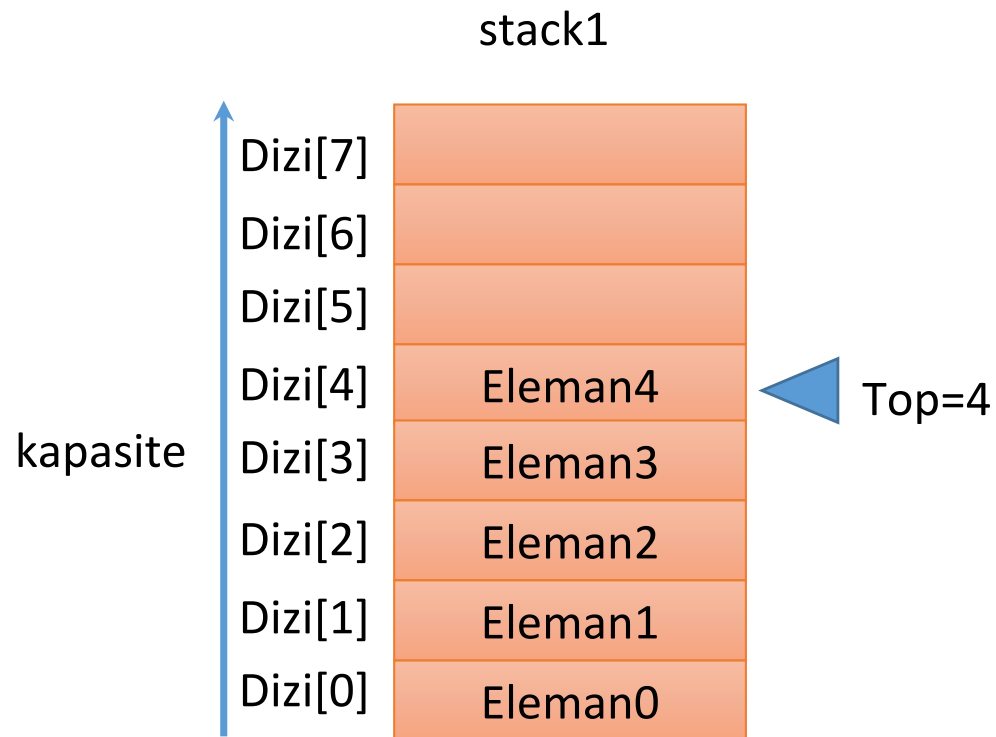


Stack – Dizi ile gerçekleştirme



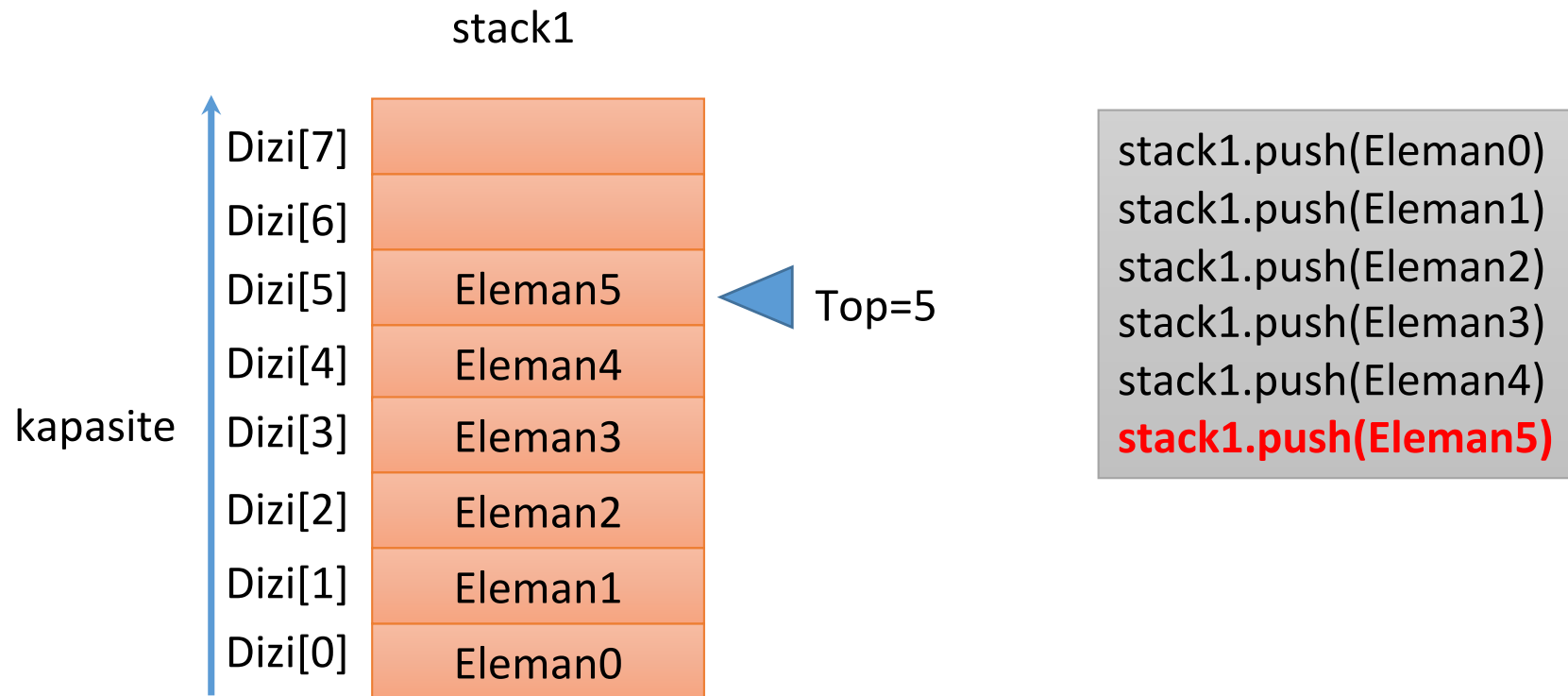
```
stack1.push(Eleman0)  
stack1.push(Eleman1)  
stack1.push(Eleman2)  
stack1.push(Eleman3)
```

Stack – Dizi ile gerçekleştirme

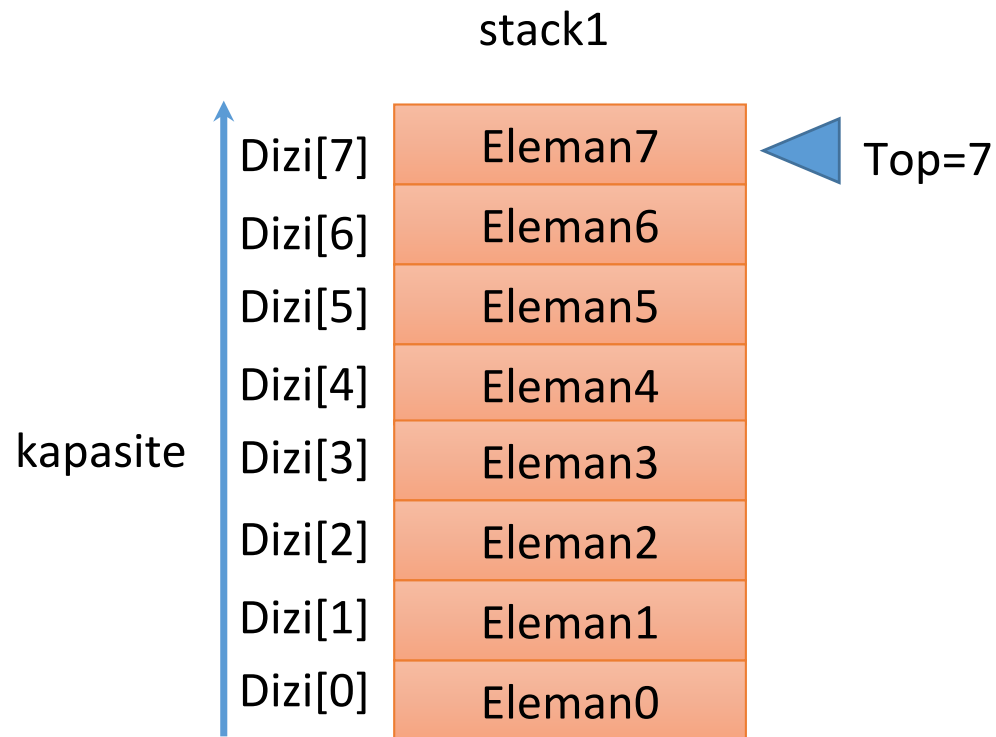


```
stack1.push(Eleman0)  
stack1.push(Eleman1)  
stack1.push(Eleman2)  
stack1.push(Eleman3)  
stack1.push(Eleman4)
```

Stack – Dizi ile gerçekleştirme

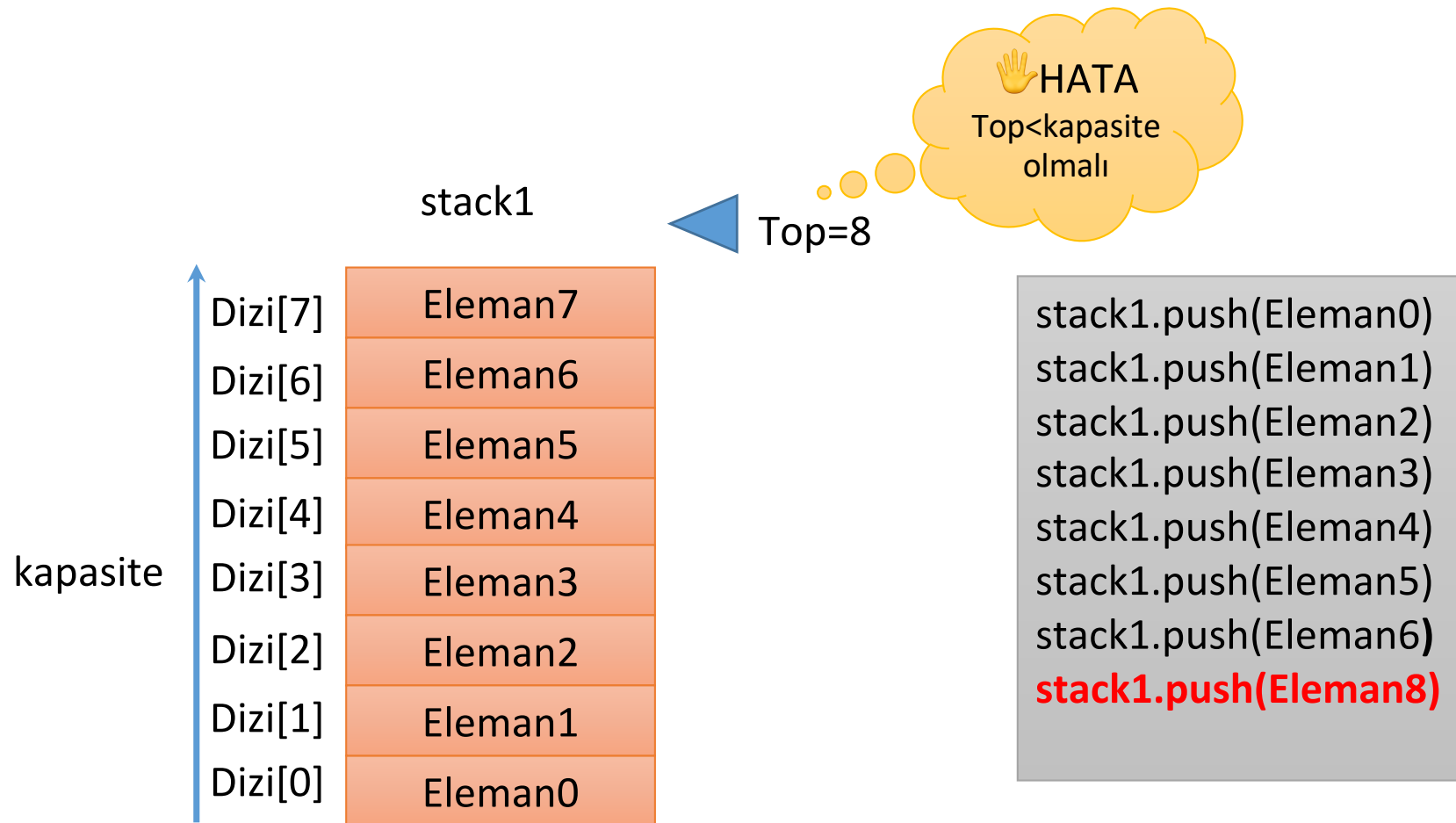


Stack – Dizi ile gerçekleştirme

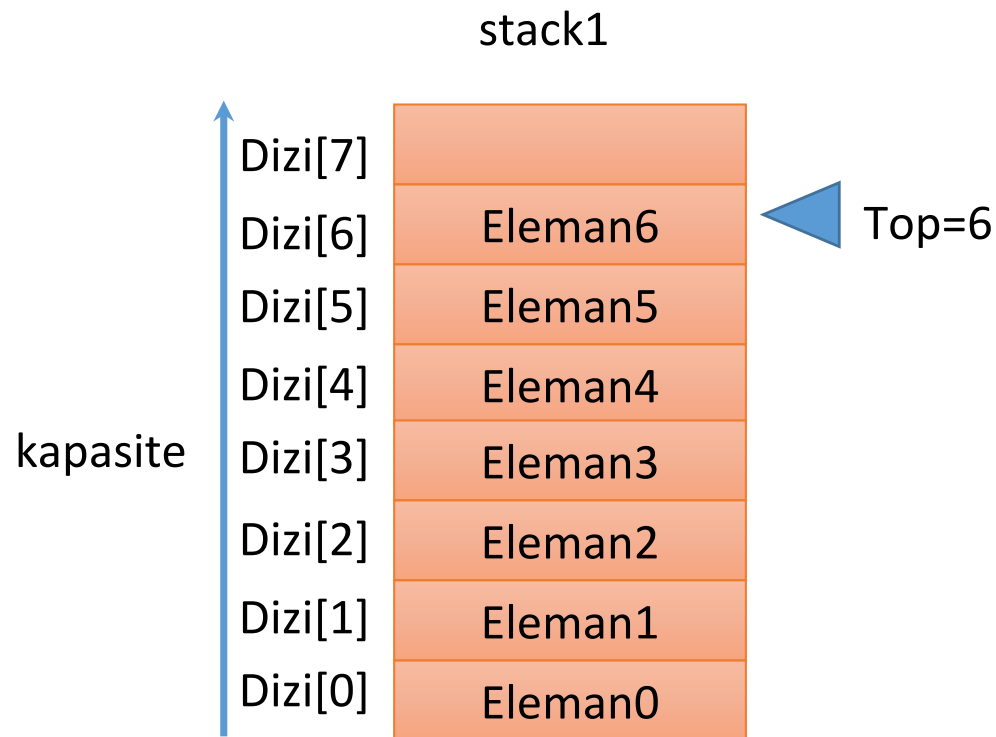


```
stack1.push(Eleman0)
stack1.push(Eleman1)
stack1.push(Eleman2)
stack1.push(Eleman3)
stack1.push(Eleman4)
stack1.push(Eleman5)
stack1.push(Eleman6)
stack1.push(Eleman7)
```

Stack – Dizi ile gerçekleştirme

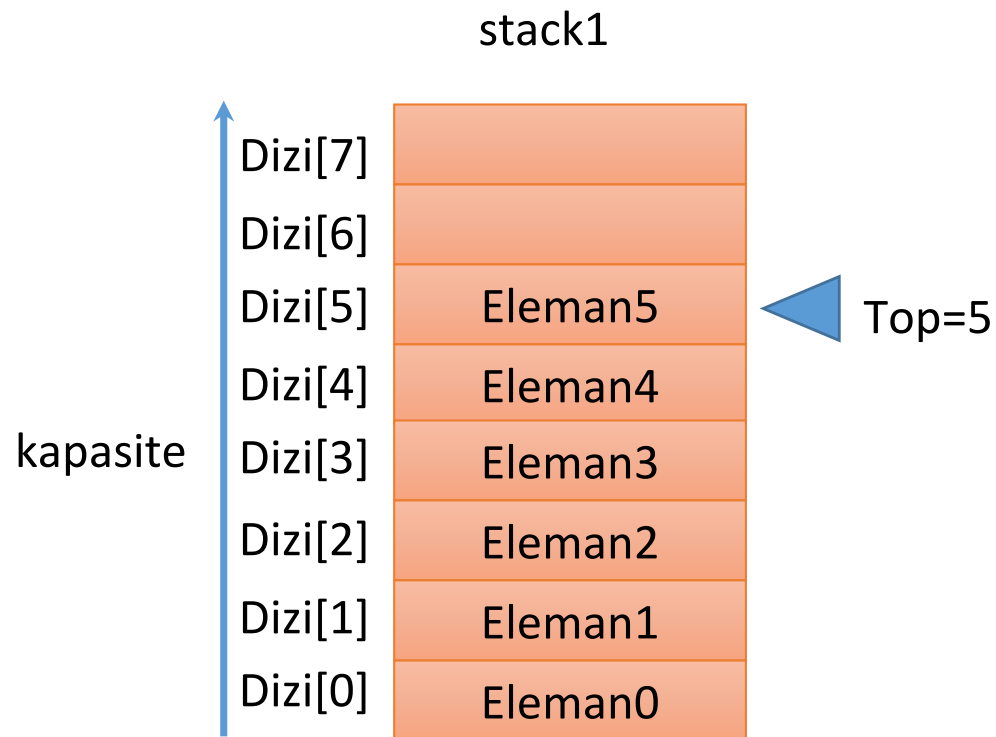


Stack – Dizi ile gerçekleştirme



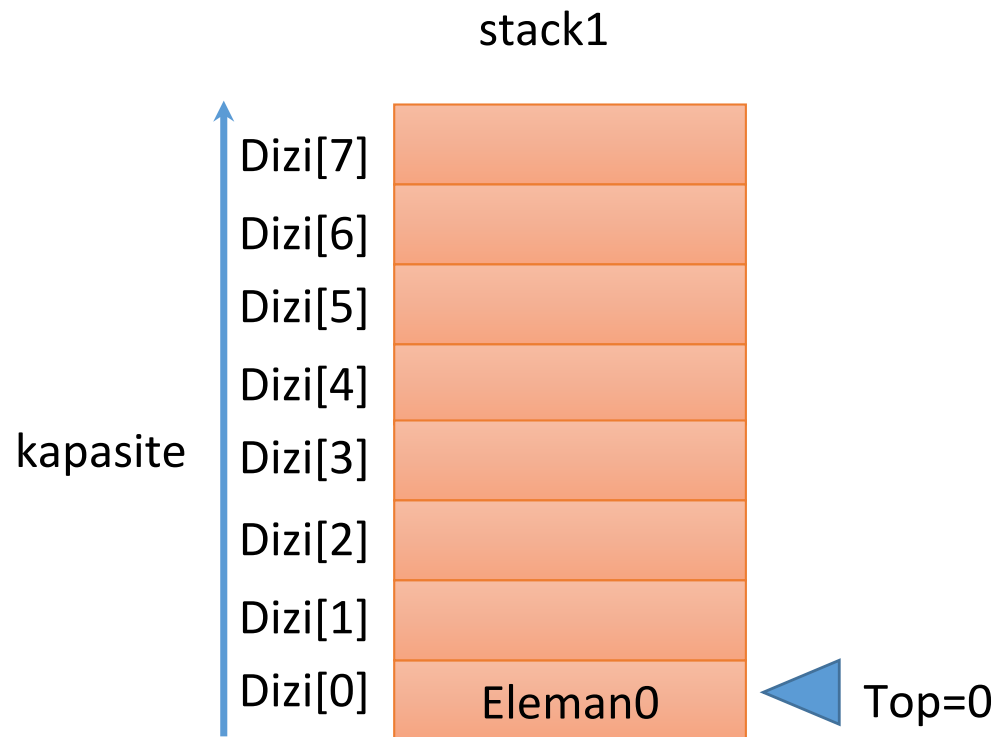
```
stack1.push(Eleman0)
stack1.push(Eleman1)
stack1.push(Eleman2)
stack1.push(Eleman3)
stack1.push(Eleman4)
stack1.push(Eleman5)
stack1.push(Eleman6)
stack1.push(Eleman7)
stack1.pop()
```

Stack – Dizi ile gerçekleştirme



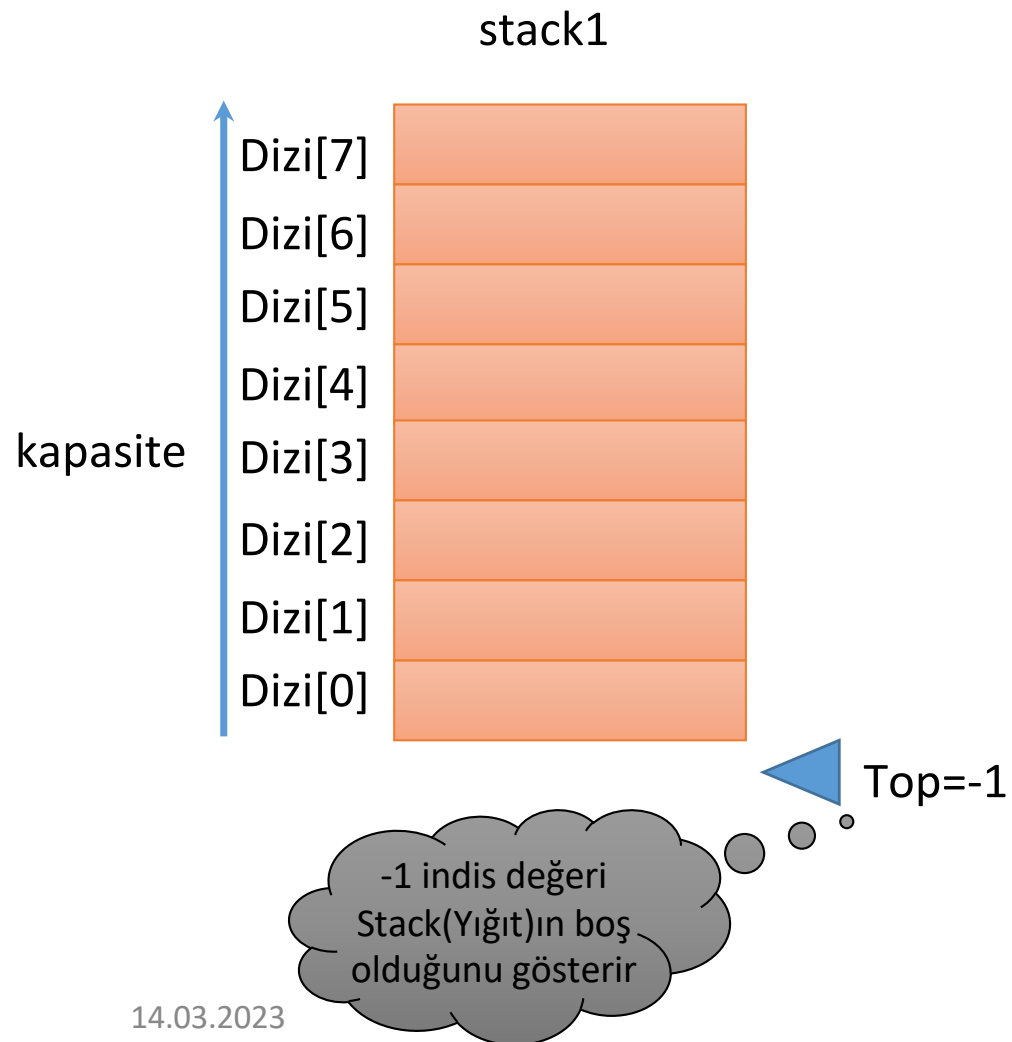
```
stack1.push(Eleman0)
stack1.push(Eleman1)
stack1.push(Eleman2)
stack1.push(Eleman3)
stack1.push(Eleman4)
stack1.push(Eleman5)
stack1.push(Eleman6)
stack1.push(Eleman7)
stack1.pop()
stack1.pop()
```

Stack – Dizi ile gerçekleştirme



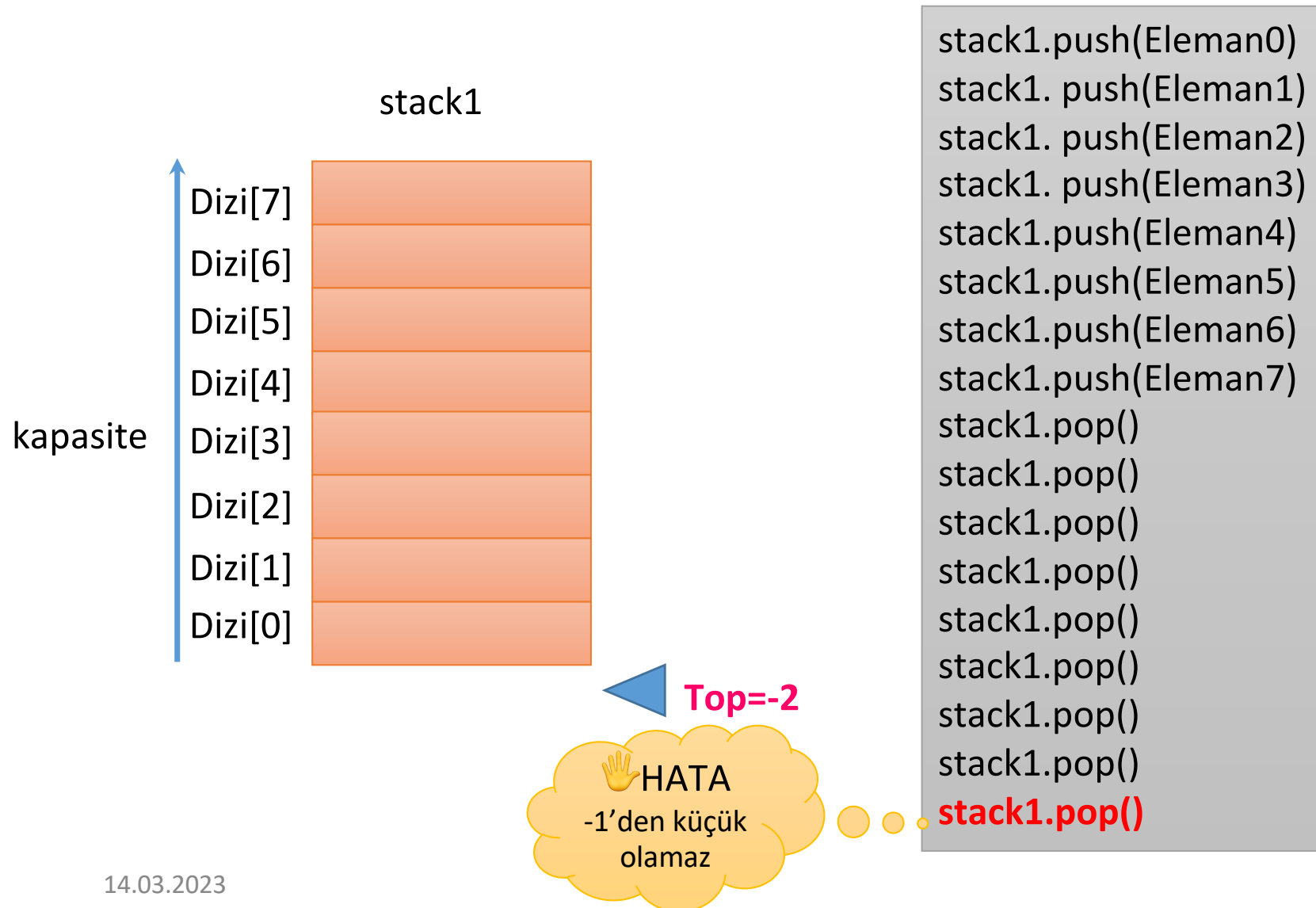
```
stack1.push(Eleman0)  
stack1.push(Eleman1)  
stack1.push(Eleman2)  
stack1.push(Eleman3)  
stack1.push(Eleman4)  
stack1.push(Eleman5)  
stack1.push(Eleman6)  
stack1.push(Eleman7)  
stack1.pop()  
stack1.pop()  
stack1.pop()  
stack1.pop()  
stack1.pop()  
stack1.pop()  
stack1.pop()
```

Stack – Dizi ile gerçekleştirme

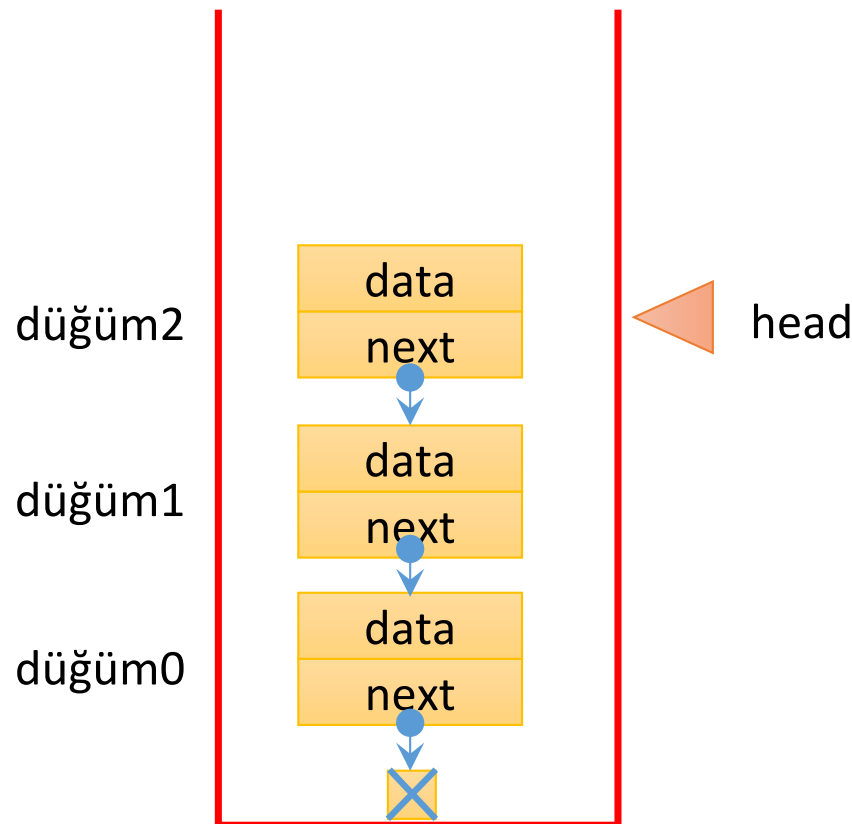


```
stack1.push(Eleman0)
stack1.push(Eleman1)
stack1.push(Eleman2)
stack1.push(Eleman3)
stack1.push(Eleman4)
stack1.push(Eleman5)
stack1.push(Eleman6)
stack1.push(Eleman7)
stack1.pop()
stack1.pop()
stack1.pop()
stack1.pop()
stack1.pop()
stack1.pop()
stack1.pop()
stack1.pop()
```

Stack – Dizi ile gerçekleştirme

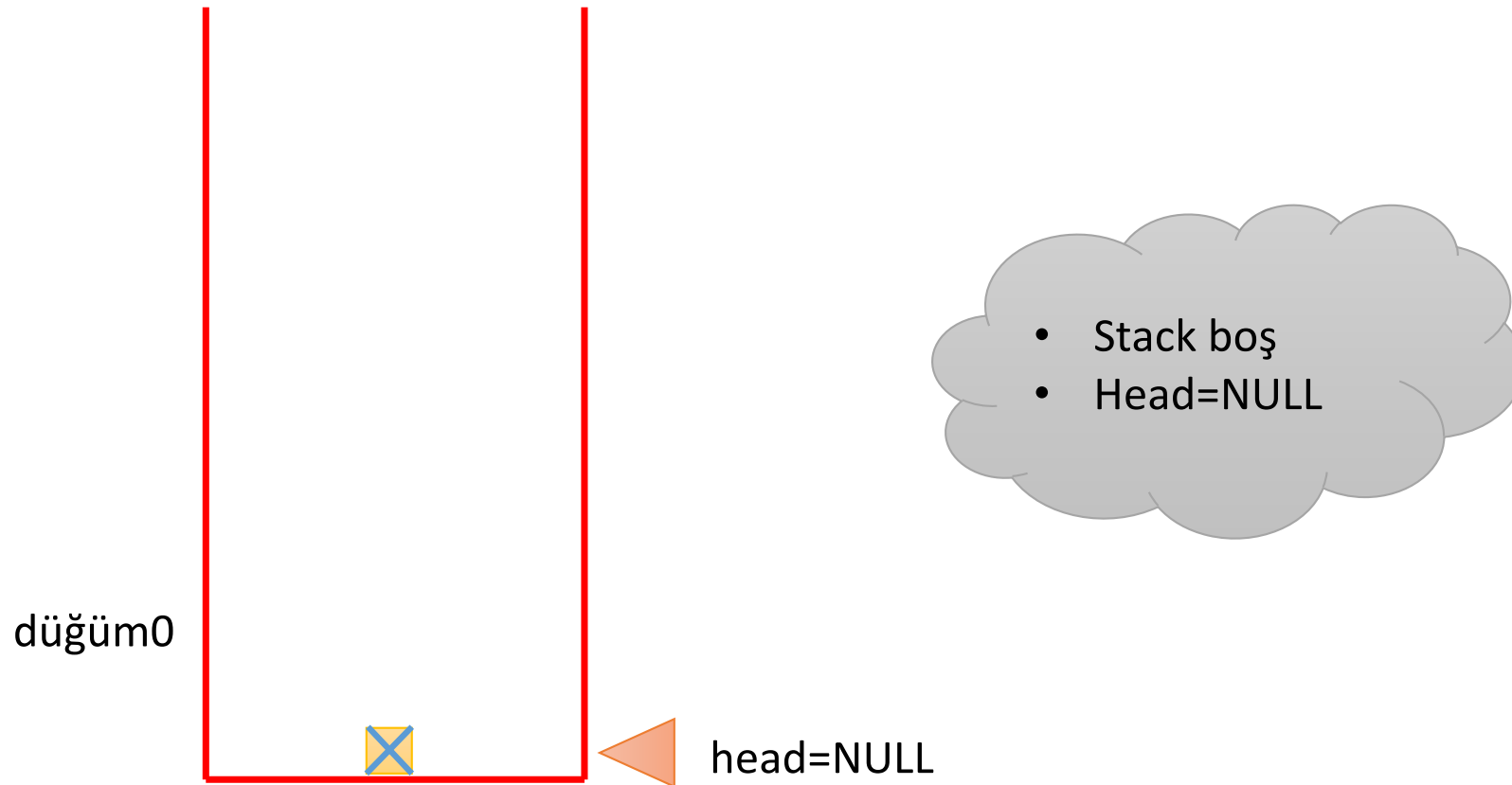


Bağlı Stack(Yığıt) Veri Yapısı

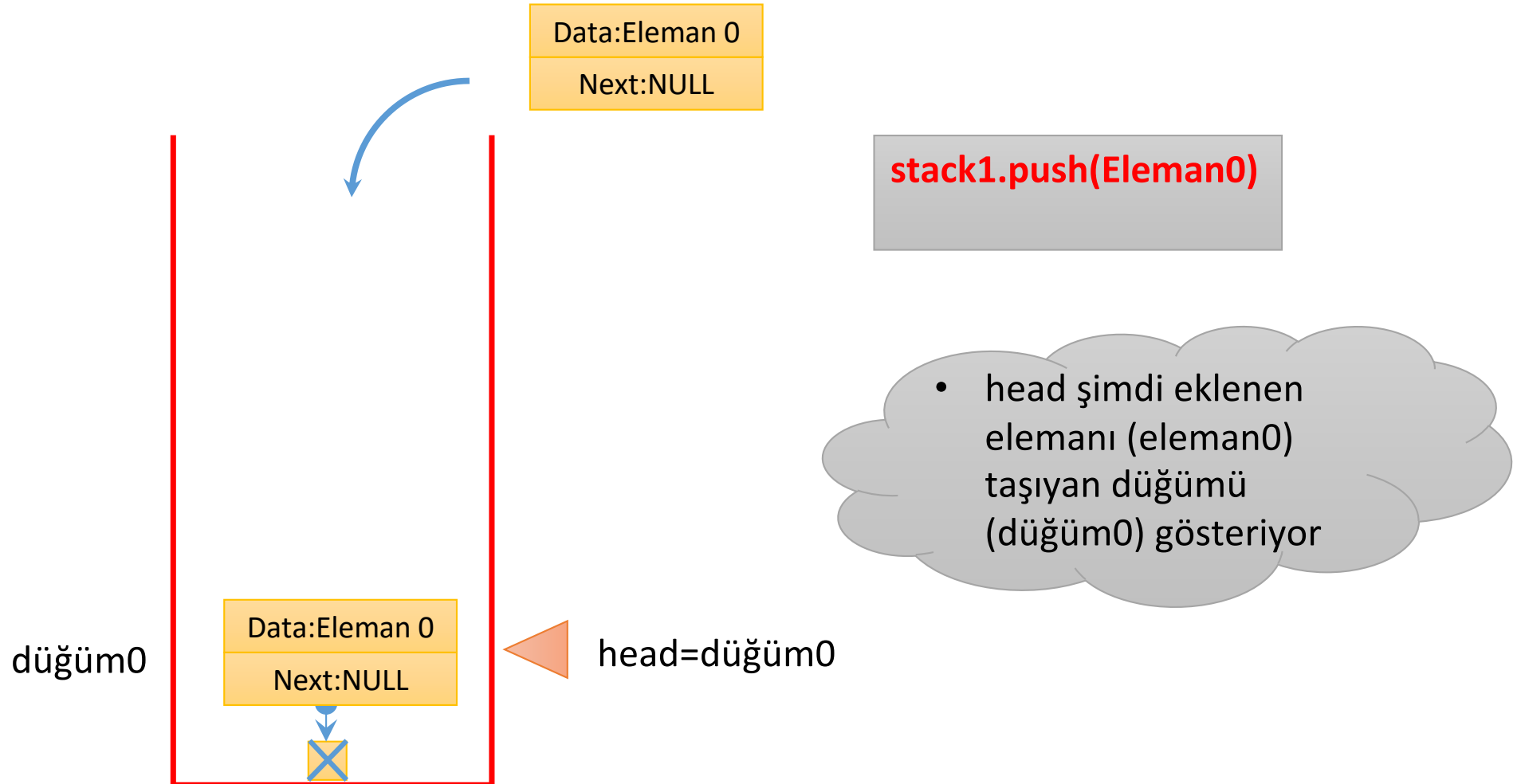


- Bağlı listeyi oluşturan düğümler, Stack(Yığıt) veri yapısını oluşturmak için kullanılabilir.
- Stack(Yığıt)' e eleman eklemek bir bağlı listenin başına eleman eklemek gibi düşünülebilir. (`push_front(eleman)`)
- Stack(Yığıt)' ten eleman silmek ise yine bağlı listenin ilk elemanını silmek gibidir. (`pop_front()`)
- Liste başı (head) Stack(Yığıt)' in tepesini gösterir.

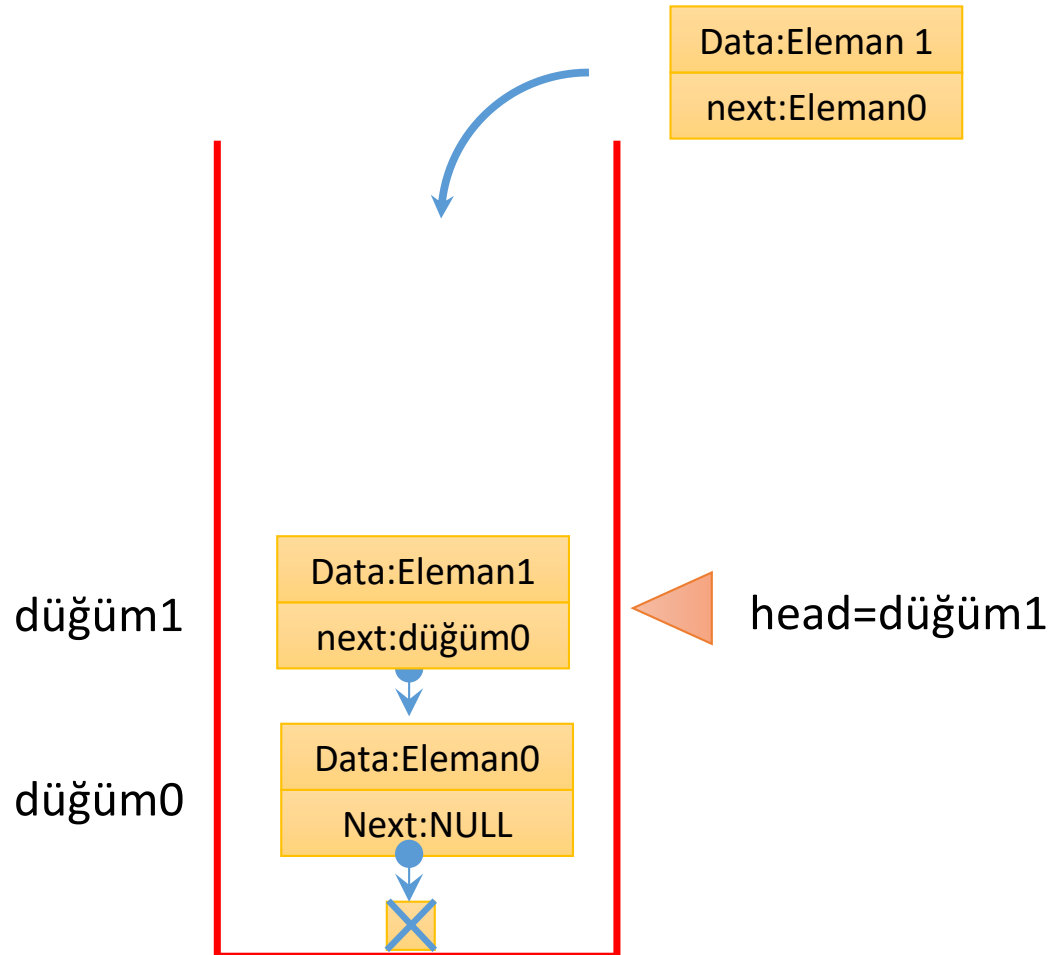
Bağlı Stack(Yığıt) Veri Yapısı



Bağlı Stack(Yığıt) Veri Yapısı



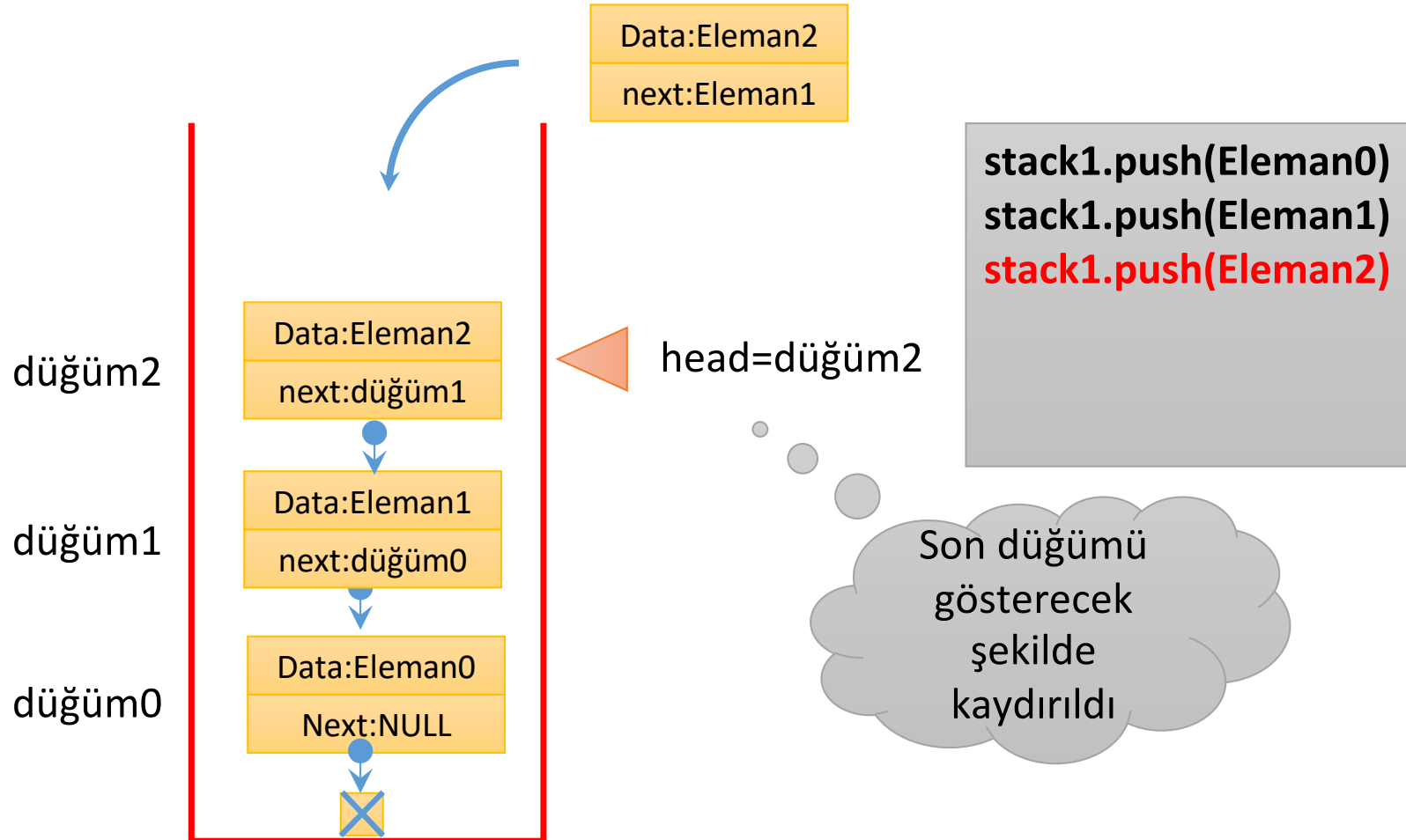
Bağlı Stack(Yığıt) Veri Yapısı



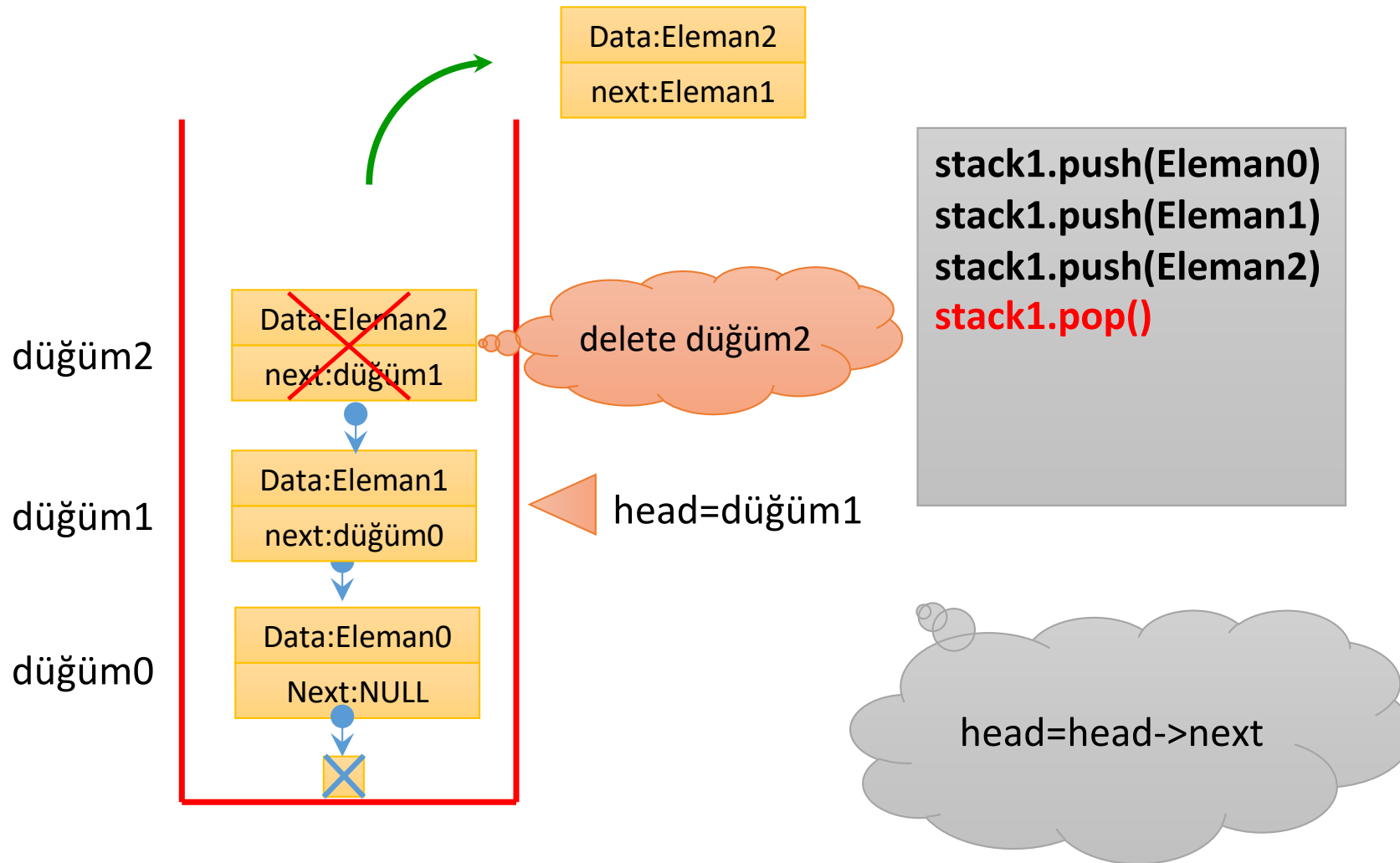
```
stack1.push(Eleman0)  
stack1.push(Eleman1)
```

Son düğümü
gösterecek
şekilde
kaydırıldı

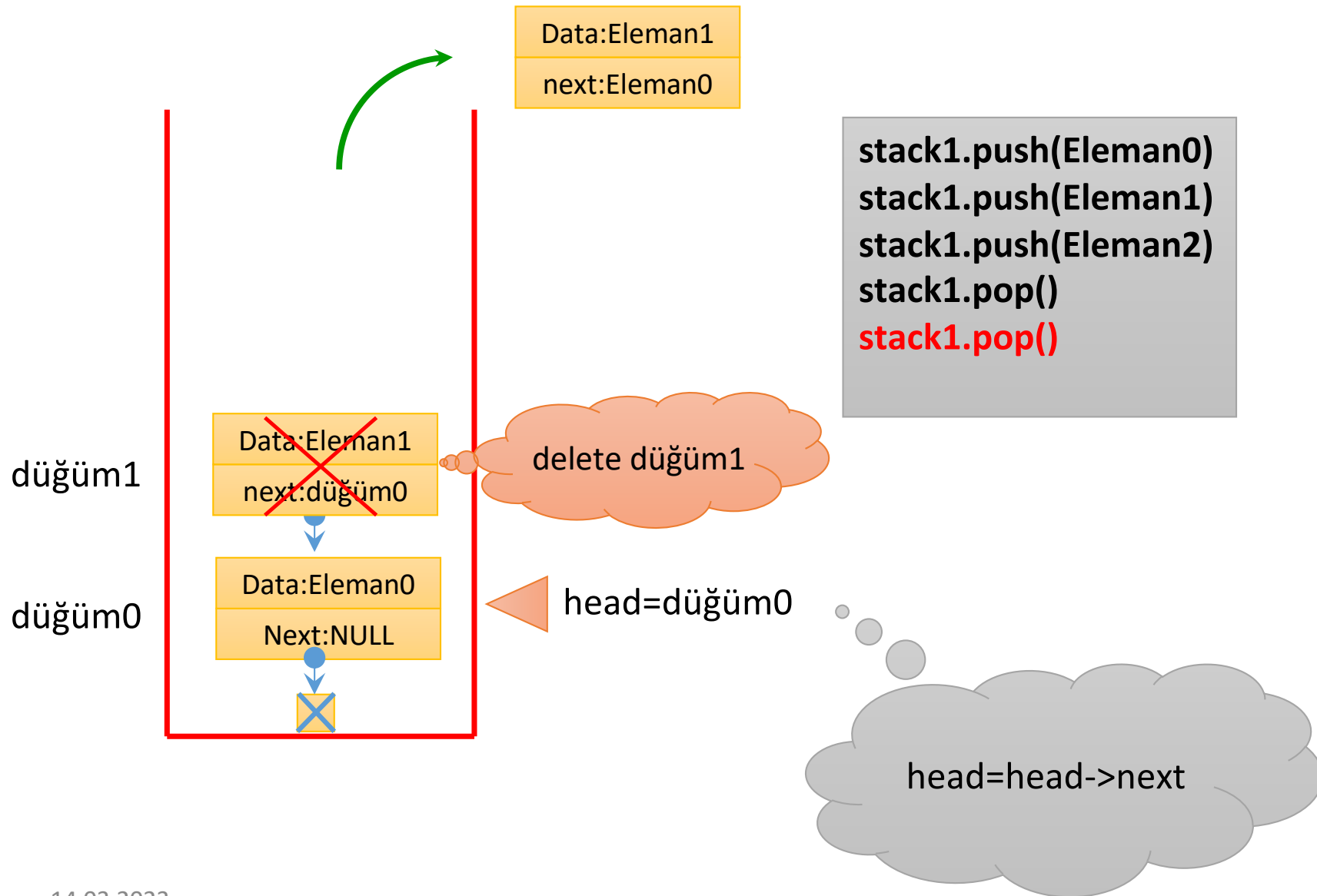
Bağlı Stack(Yığıt) Veri Yapısı



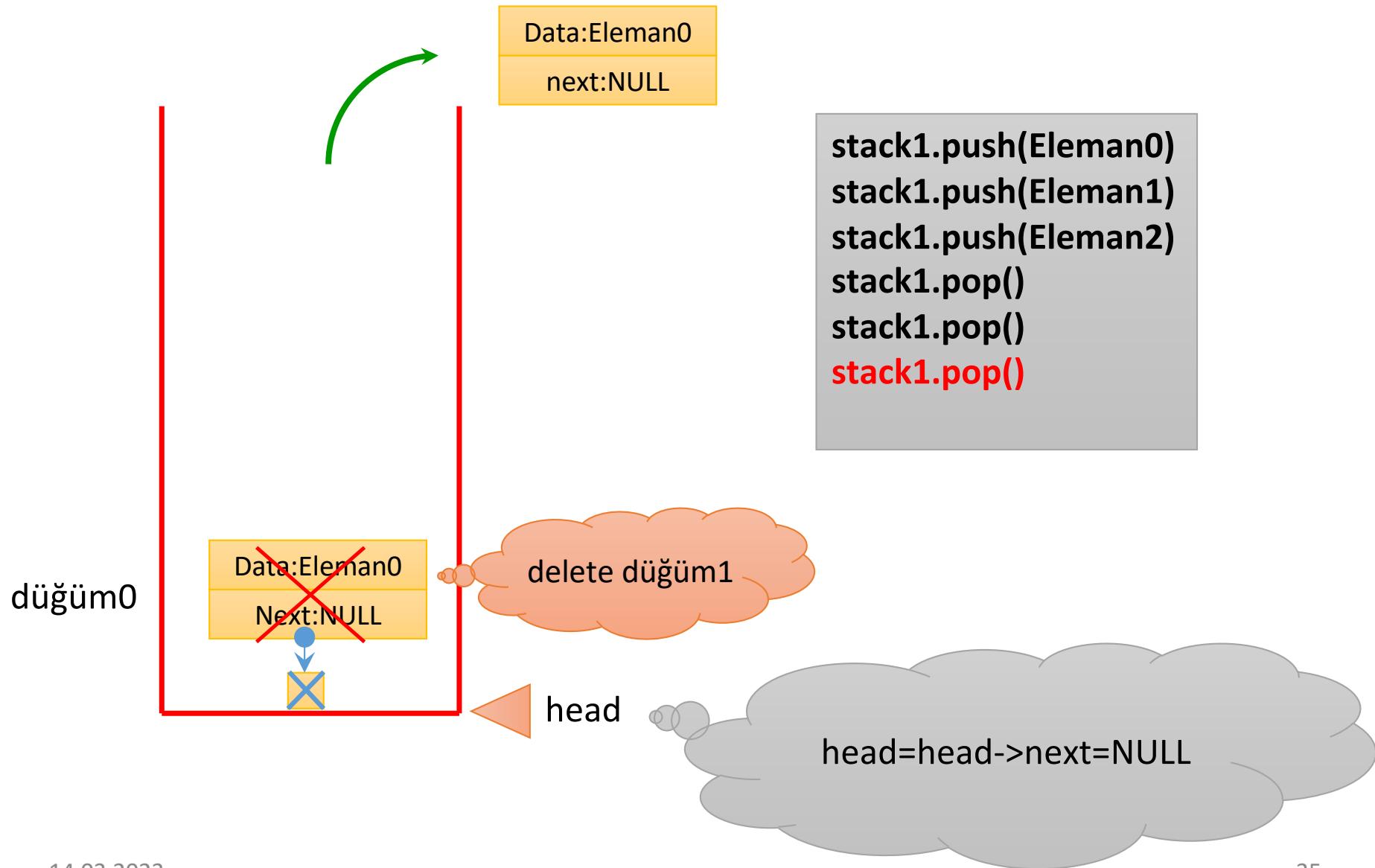
Bağlı Stack(Yığıt) Veri Yapısı



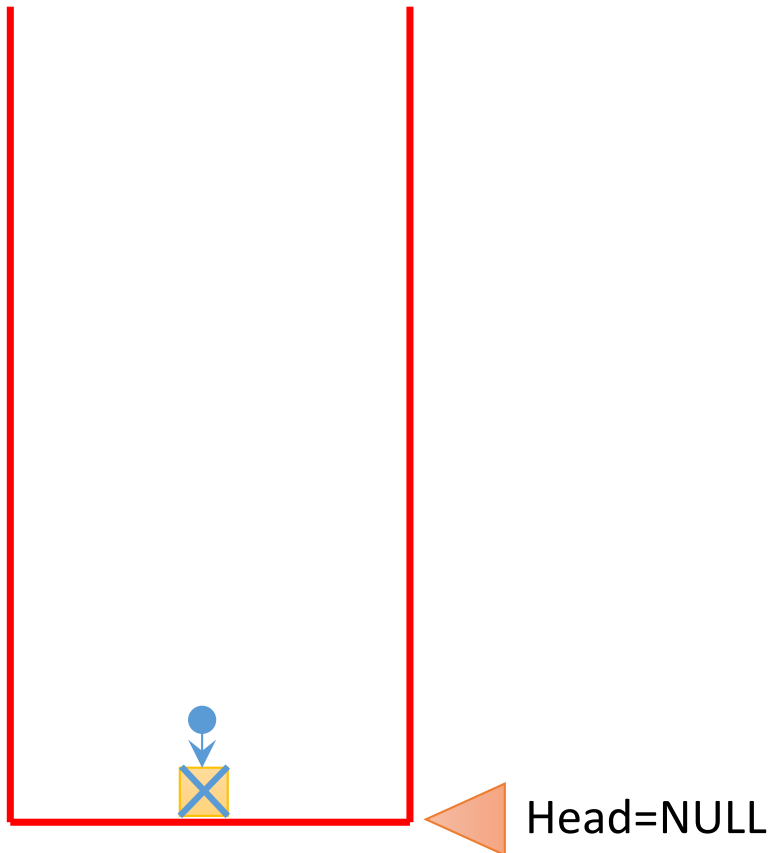
Bağlı Stack(Yığıt) Veri Yapısı



Bağlı Stack(Yığıt) Veri Yapısı



Bağlı Stack(Yığıt) Veri Yapısı



```
stack1.push(Eleman0)  
stack1.push(Eleman1)  
stack1.push(Eleman2)  
stack1.pop()  
stack1.pop()  
stack1.pop()
```

Sentaks Kontrolü

- Açılan – Kapanan parantezlerin karşılaştırılması
 - Açılan parantez kapatılmazsa hata oluşur.
 - Açılan her parantez için yığta veri eklenir.
 - Kapanan her parantez için yığıttan veri çıkarılır.

Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```

Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```

Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```

Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x ) { if  
  ( theItems.length == size ( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x ) { if  
  ( theItems.length == size ( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x){ if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



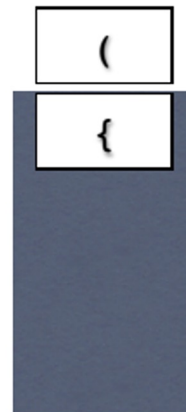
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
    2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
    [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
    +; }
```



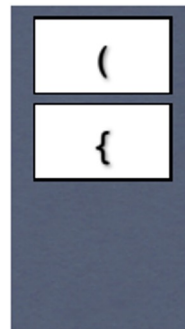
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
    2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
    [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
    +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
    * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
    [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
    +; }
```



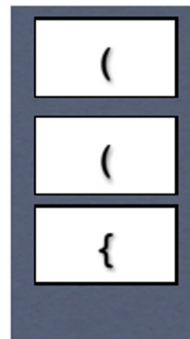
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



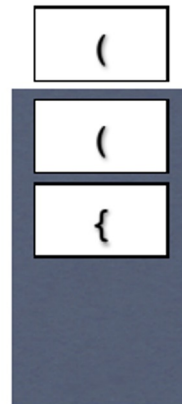
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



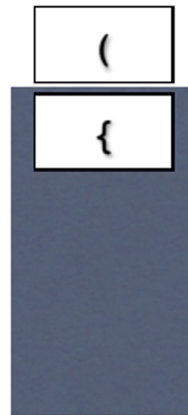
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



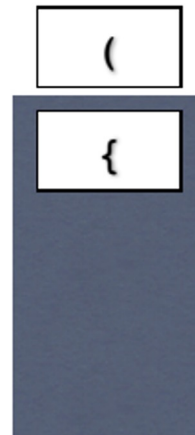
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



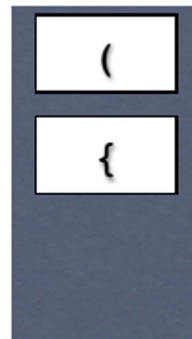
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



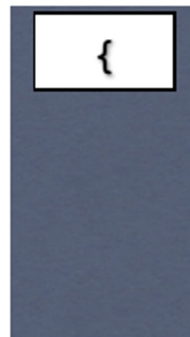
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1 ); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



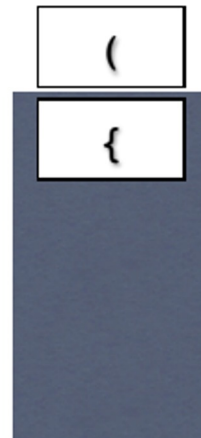
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



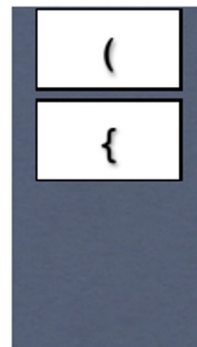
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



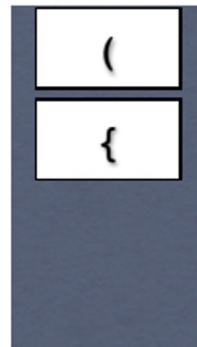
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



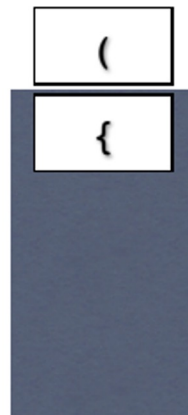
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



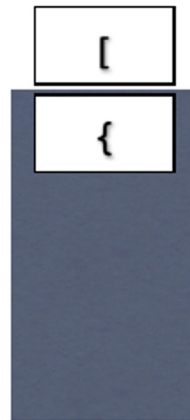
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



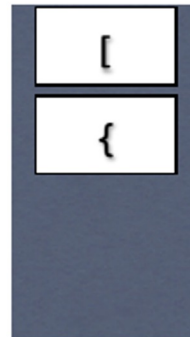
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  + 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  ; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  ; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



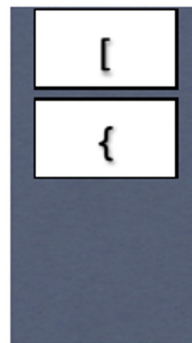
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



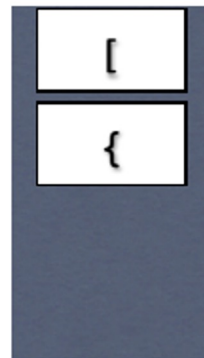
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



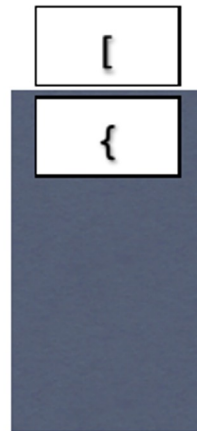
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1]; theItems[ idx ] = x; theSize+  
  +; }
```



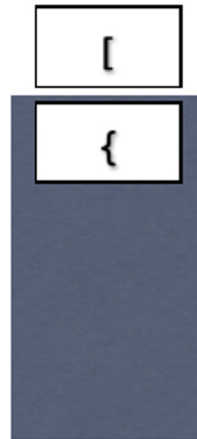
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1]; theItems[ idx ] = x; theSize+  
  +; }
```



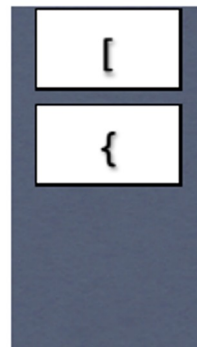
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[idx] = x; theSize+  
  +; }
```



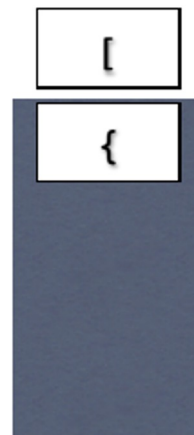
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



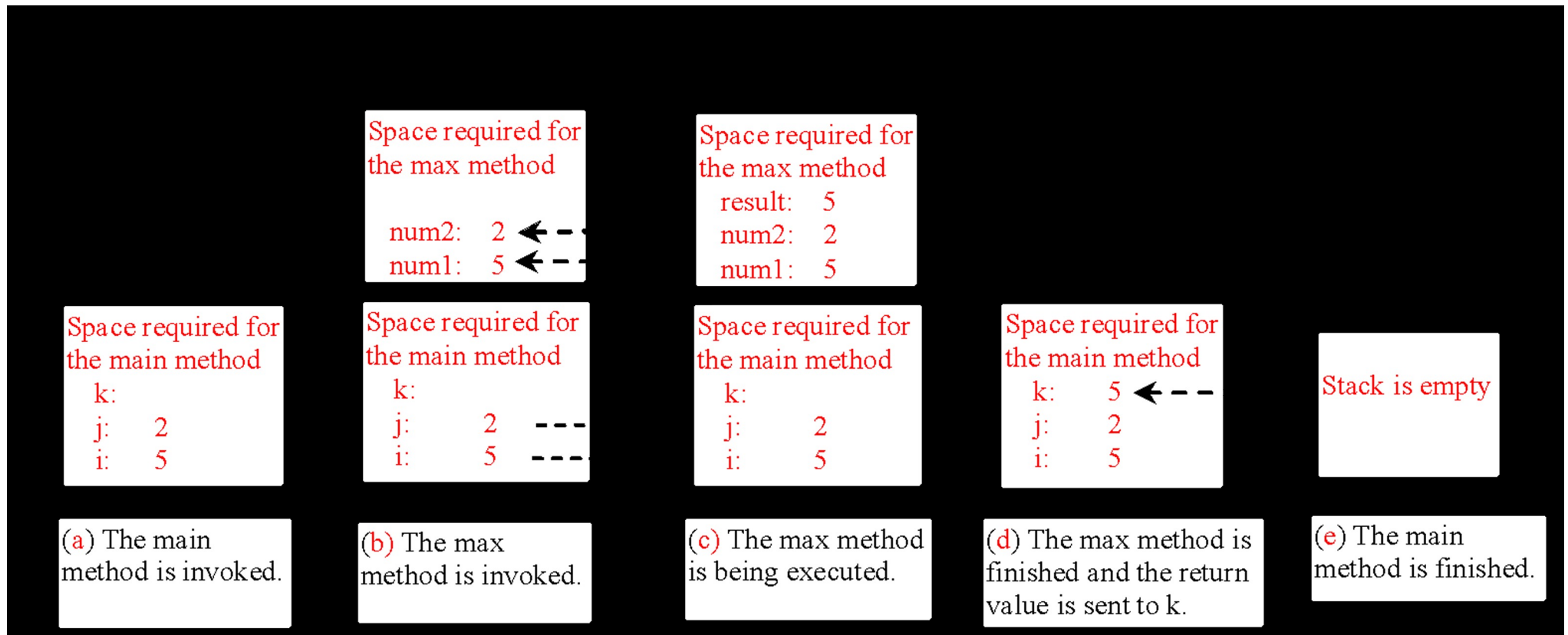
Syntax Checking

```
* public void add( int idx, AnyType x) { if  
  ( theItems.length == size( ) ) ensureCapacity( size ( )  
  * 2 + 1); for( int i=theSize; i > idx; i-- ) theItems  
  [ i ] = theItems[ i - 1 ]; theItems[ idx ] = x; theSize+  
  +; }
```



Bellek Alanının Stack ile Yönetimi

- Bir program ve alt programlarının çalışma anındaki bellek ihtiyaçları stack ile yönetilebilir



Bellek Alanının Stack ile Yönetimi

i tanımlandı ve başlatıldı

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

i: 5

The main method
is invoked.

Bellek Alanının Stack ile Yönetimi

j tanımlandı ve başlatıldı

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

j: 2
i: 5

The main method
is invoked.

Bellek Alanının Stack ile Yönetimi

K tanımlandı

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

The main method
is invoked.

Bellek Alanının Stack ile Yönetimi

max(i, j) metodu çağırıldı

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:
j: 2
i: 5

The main method
is invoked.

Bellek Alanının Stack ile Yönetimi

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

i ve j değerleri num1 ve num2'ye aktarıldı

num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Bellek Alanının Stack ile Yönetimi

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Result tanımlandı

result:
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Bellek Alanının Stack ile Yönetimi

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

(num1 > num2) : true

result:
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Bellek Alanının Stack ile Yönetimi

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2)  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

num1 değerini result'a atadı.

Space required for the
max method

result: 5
num2: 2
num1: 5

Space required for the
main method

k:
j: 2
i: 5

The max method is
invoked.

Bellek Alanının Stack ile Yönetimi

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);  
  
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

result değerini döndür ve k
değişkenine ata

Space required for the
max method

result: 5
num2: 2
num1: 5

Space required for the
main method

k: 5
j: 2
i: 5

The max method is
invoked.

Bellek Alanının Stack ile Yönetimi

Yazdırma ifadesini icra et

```
public static void main(String[] args) {  
    int i = 5;  
    int j = 2;  
    int k = max(i, j);
```

```
    System.out.println(  
        "The maximum between " + i +  
        " and " + j + " is " + k);  
}
```

```
public static int max(int num1, int num2) {  
    int result;  
  
    if (num1 > num2)  
        result = num1;  
    else  
        result = num2;  
  
    return result;  
}
```

Space required for the
main method

k:5
j: 2
i: 5

The main method
is invoked.

Infix – Postfix – Prefix Notasyonları

- Infix
 - Günlük matematik işlemlerinde kullanılan notasyon
 - İşlem operatörü operandların arasında
 - $A + B * C$
- Prefix
 - İşlem operatörü operandların önünde
 - $+ A * B C$
- Postfix
 - İşlem operatörü operandların arkasında
 - $A B C * +$

Infix – Postfix – Prefix Notasyonları

Infix	Prefix	Postfix
$(A + B) * C$	$* + A B C$	$A B + C *$
$A + B * C + D$	$++ A * B C D$	$A B C * + D +$
$(A + B) * (C + D)$	$* + A B + C D$	$A B + C D + *$
$A * B + C * D$	$+ * A B * C D$	$A B * C D * +$
$A + B + C + D$	$+++ A B C D$	$A B + C + D +$

Infix ifadeyi Postfix ifadeye Dönüştürme

- Post-fix notasyonuna geçişte, 4 durumu kontrol ederiz.
 - ❖ Okunan karakter aç parantez ise,
 - ❖ Okunan karakter kapa parantez ise,
 - ❖ Okunan karakter operatör ise,
 - ❖ Okunan karakter operand ise,

Stack uygulaması: Infix, Prefix, Postfix

- **Infix notasyonunda**

- A ve B yi toplamak için

$$A+B$$

- A ve B'yi çarpmak için

$$A*B$$

- Operatörler (*,+, -, /) operandların (A,B) arasına gelir

Infix, Prefix, Postfix

- **Prefix notasyonunda**
- iki operand arasındaki operatör önce belirtilir.
- A ve B yi toplamak için, topla A B
+ A B
- A ve B'yi çarpmak için, çarp A B
*** A B**

Infix, Prefix, Postfix

- **Postfix notasyonunda**
- iki operand arasındaki operatör sonra belirtilir.
- A ve B yi toplamak için, A B topla

A B +

- A ve B'yi çarpmak için, A B çarp

A B *

Infix, Prefix, Postfix

- Parantezler
- $5+6*7$ ifadesinin hesaplanması
- Önce toplama:
$$(5+6)*7 = 11*7 = 77$$
- Önce çarpma:
$$5+(6*7) = 5+42 = 47$$
- Parantezlerin kullanımı infix notasyonunda önemlidir.

Infix, Prefix, Postfix

- **Prefix notasyonu**

- $+ 5 * 6 7 =$

$$= + 5 42$$

$$= 47$$

- $* + 5 6 7 =$

$$= * 11 7$$

$$= 77$$

- Parantez kullanmadan işlem önceliği tanımlanabilir

Infix, Prefix, Postfix

- **Postfix notasyonu**

- $5 \ 6 \ 7 \ * \ + \ =$

$$= 5 \ 42 \ +$$

$$= 47$$

- $5 \ 6 \ + \ 7 \ * \ =$

$$= 11 \ 7 \ *$$

$$= 77$$

- Parantez kullanmadan işlem önceliği tanımlanabilir

Örnek : Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : (

- Çıkış :

Örnek: Infix'den postfix'e dönüşüm

- $(\mathbf{10}+20)*(30+40)/(50+60)$

- Stack : (

- Çıkış : 10

Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : (+

- Çıkış : 10

Örnek: Infix'den postfix'e dönüşüm

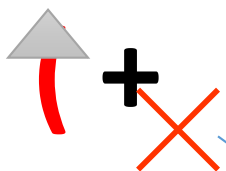
- $(10+20)*(30+40)/(50+60)$

- Stack : (+

- Çıkış : 10 20

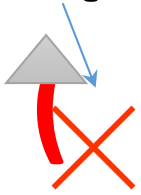
Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : 
- Çıkış : 10 20 

Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : 

- Çıkış : 10 20 +

Örnek: Infix'den postfix'e dönüşüm

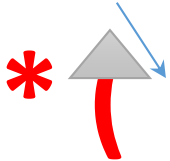
- $(10+20) * (30+40) / (50+60)$

- Stack : 

- Çıkış : 10 20 +

Örnek: Infix'den postfix'e dönüşüm

- $(10+20) * (30+40) / (50+60)$

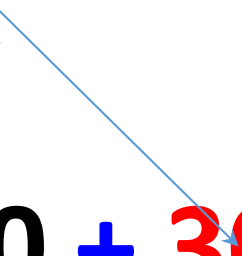
- Stack : * 

- Çıkış : 10 20 +

Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : * (

- Çıkış : 10 20 + 30 

Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : * (+ 

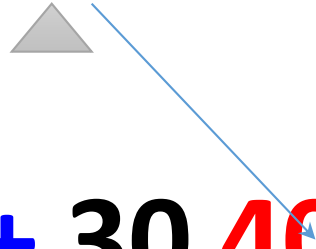
- Çıkış : 10 20 + 30

Örnek: Infix'den postfix'e dönüşüm

• $(10+20)*(30+40)/(50+60)$

• Stack : * (+

• Çıkış : 10 20 + 30 40



Örnek: Infix'den postfix'e dönüşüm

• $(10+20)*(30+40)/(50+60)$

• Stack : * (+ 

• Çıkış : 10 20 + 30 40 + 

Örnek: Infix'den postfix'e dönüşüm

• $(10+20)*(30+40)/(50+60)$

• Stack : * (

• Çıkış : 10 20 + 30 40 +

Örnek: Infix'den postfix'e dönüşüm

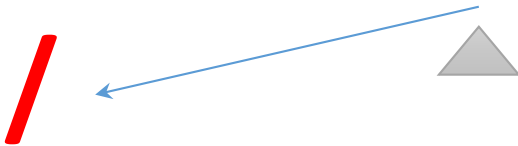
• $(10+20)*(30+40)/(50+60)$

• Stack : 

• Çıkış : 10 20 + 30 40 + * 

Örnek: Infix'den postfix'e dönüşüm

• $(10+20)*(30+40)/(50+60)$

• Stack : 

• Çıkış : 10 20 + 30 40 + *

Örnek: Infix'den postfix'e dönüşüm

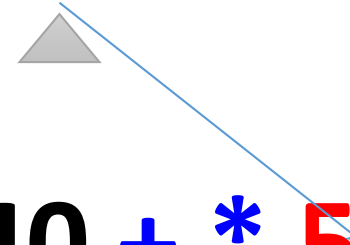
- $(10+20)*(30+40)/(50+60)$

- Stack : $/ ($ 

- Çıkış : $10\ 20\ +\ 30\ 40\ +\ *$

Örnek: Infix'den postfix'e dönüşüm

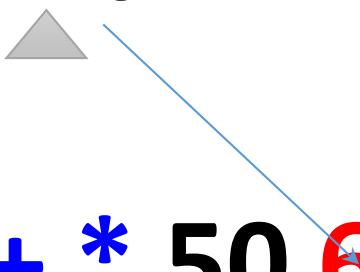
- $(10+20)*(30+40)/(50+60)$
- Stack : $/ ($
- Çıkış : 10 20 + 30 40 + * 50



Örnek: Infix'den postfix'e dönüşüm

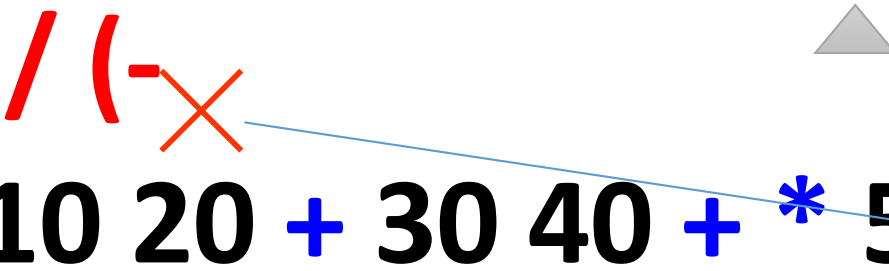
- $(10+20)*(30+40)/(50+60)$
- Stack : $/ (+$ 
- Çıkış : 10 20 + 30 40 + * 50

Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$
 - Stack : $/ (+$
 - Çıkış : 10 20 + 30 40 + * 50 60
- 

Örnek: Infix'den postfix'e dönüşüm

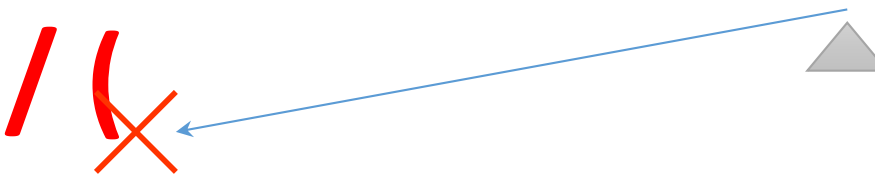
- $(10+20)*(30+40)/(50+60)$

- Stack : $/ (-$ 

- Çıkış : 10 20 + 30 40 + * 50 60 +

Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : 

- Çıkış : 10 20 + 30 40 + * 50 60 +

Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$

- Stack : /

- Çıkış : 10 20 + 30 40 + * 50 60 + /

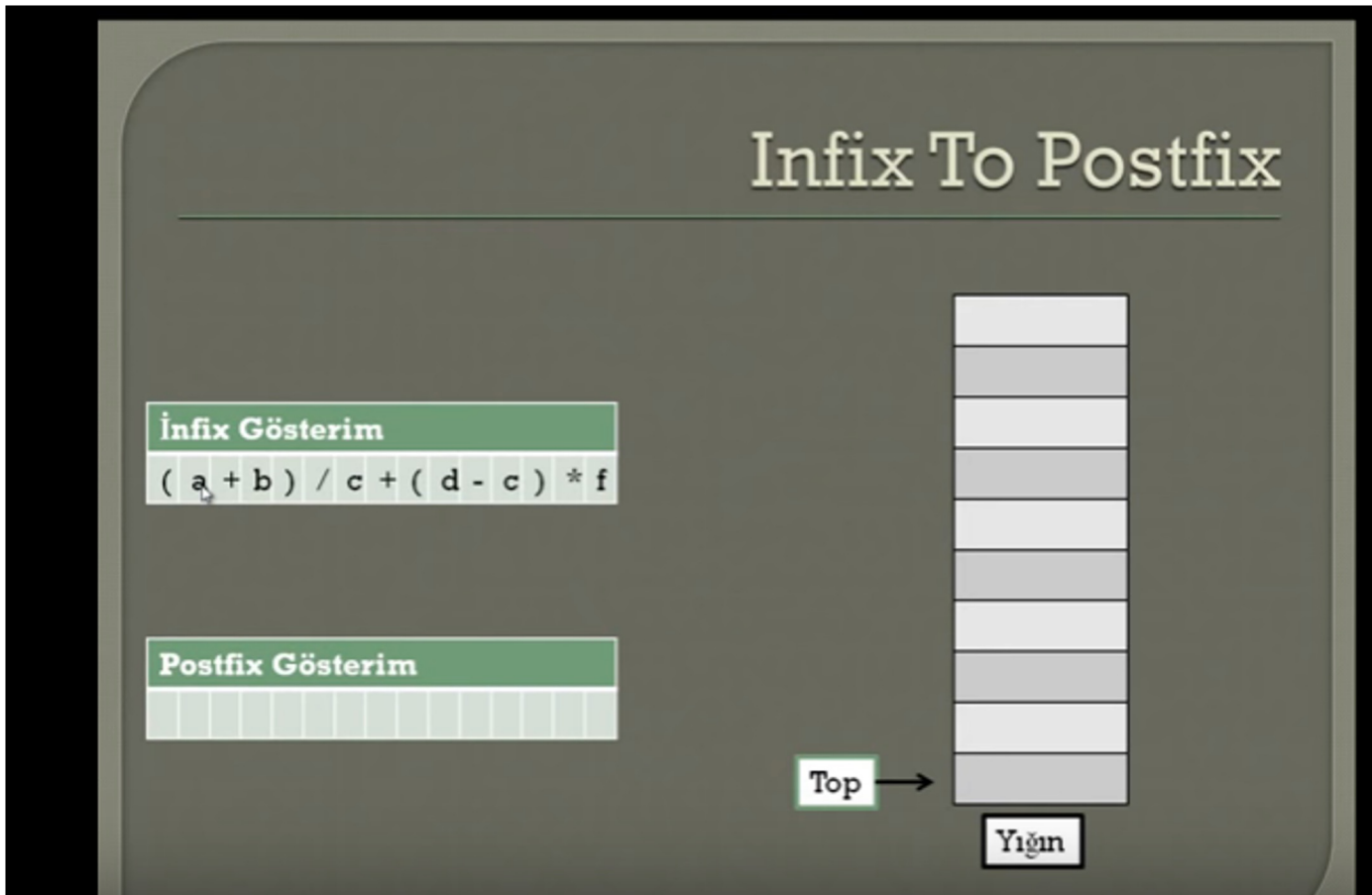
Örnek: Infix'den postfix'e dönüşüm

- $(10+20)*(30+40)/(50+60)$
- Stack :
- Çıkış : 10 20 + 30 40 + * 50 60 + /

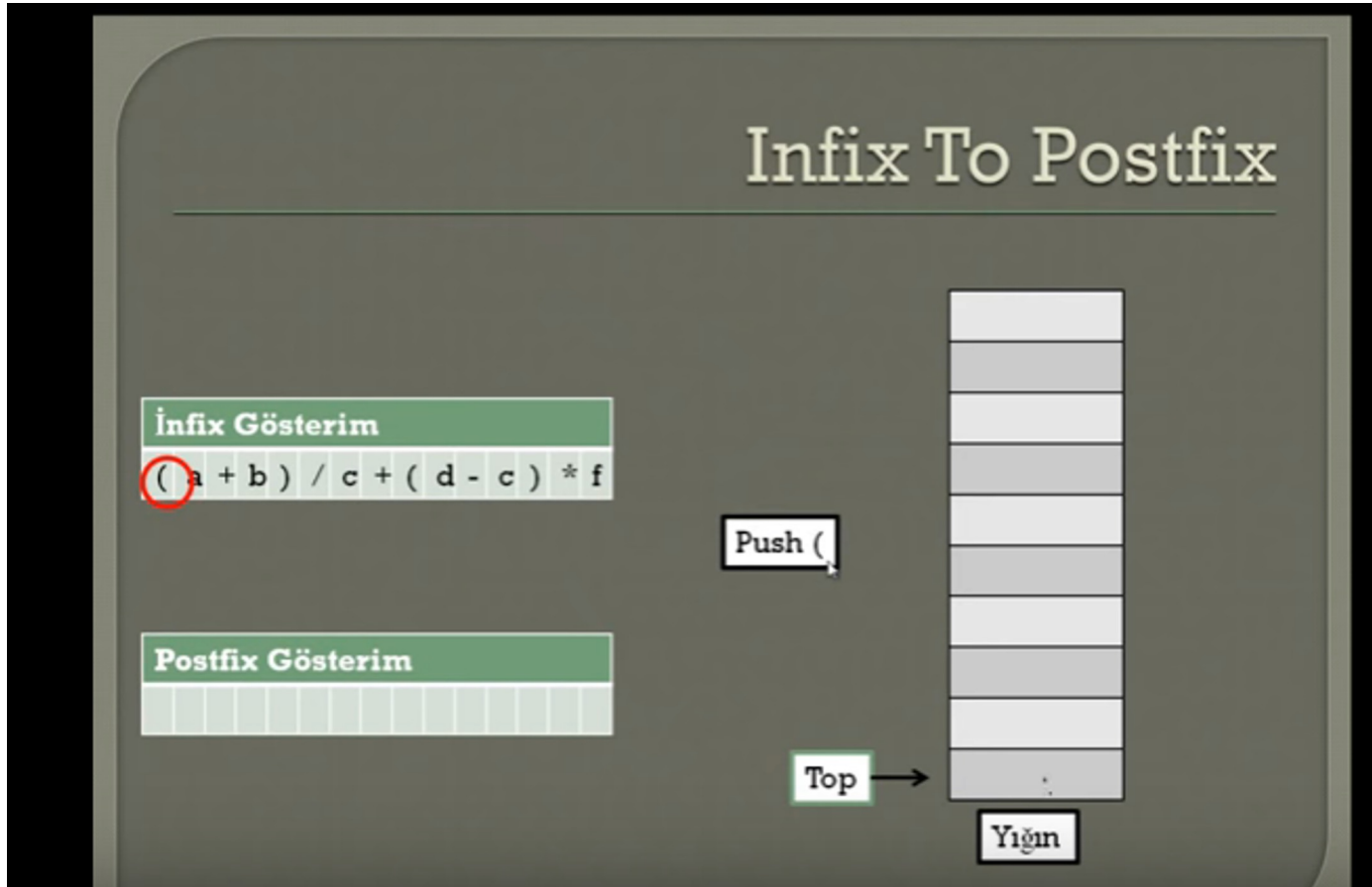


✓ Dönüşüm
tamamlandı

Infix ifadeyi Postfix İfadeye Dönüştürme

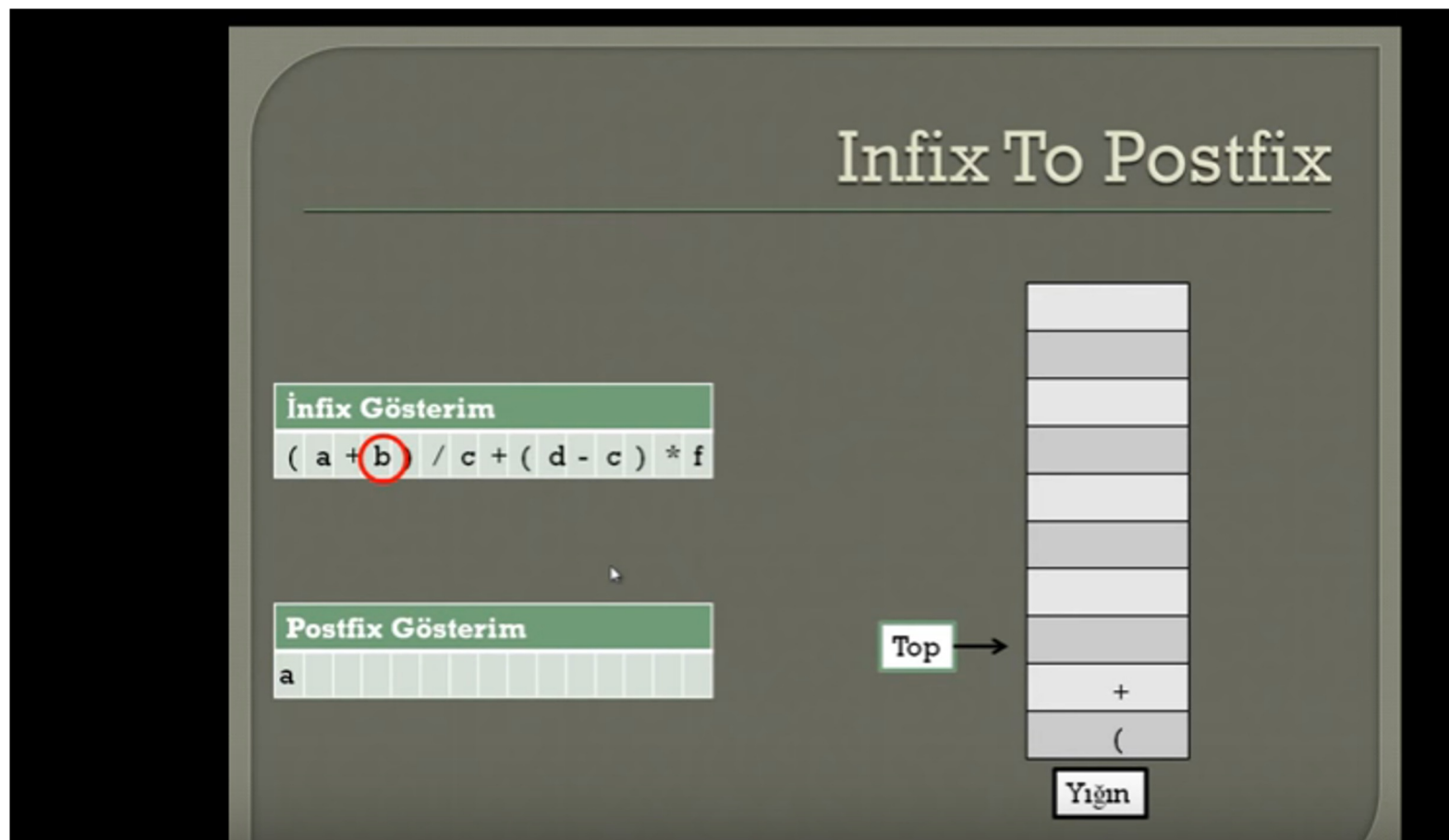


Infix ifadeyi Postfix ifadeye Dönüştürme



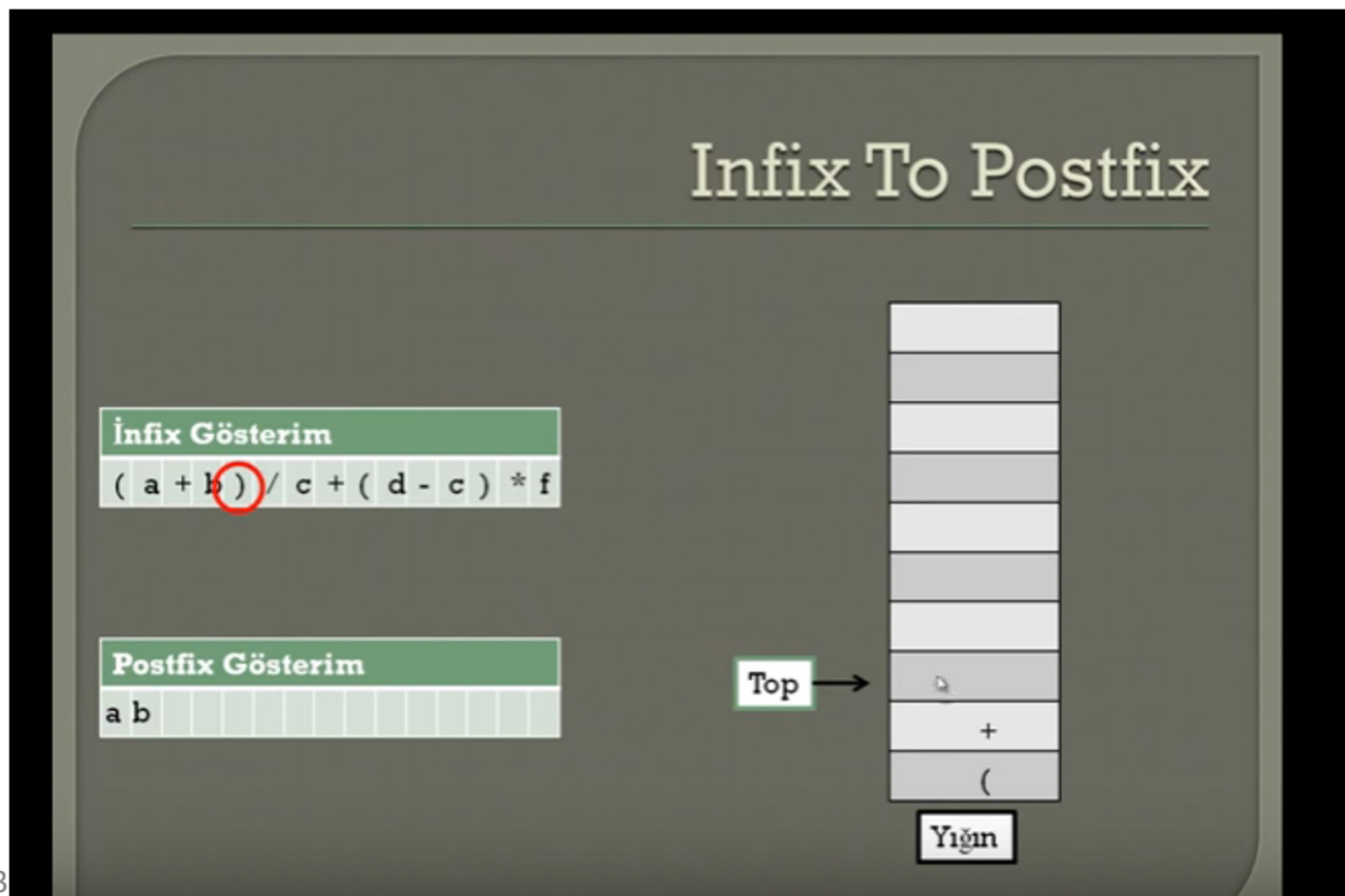
Infix ifadeyi Postfix ifadeye Dönüştürme

- Eğer stack'teki operatör'ün önceliği okunan operatörün önceliğinden büyük veya eşit ise stack'teki operatör pop() edilir.

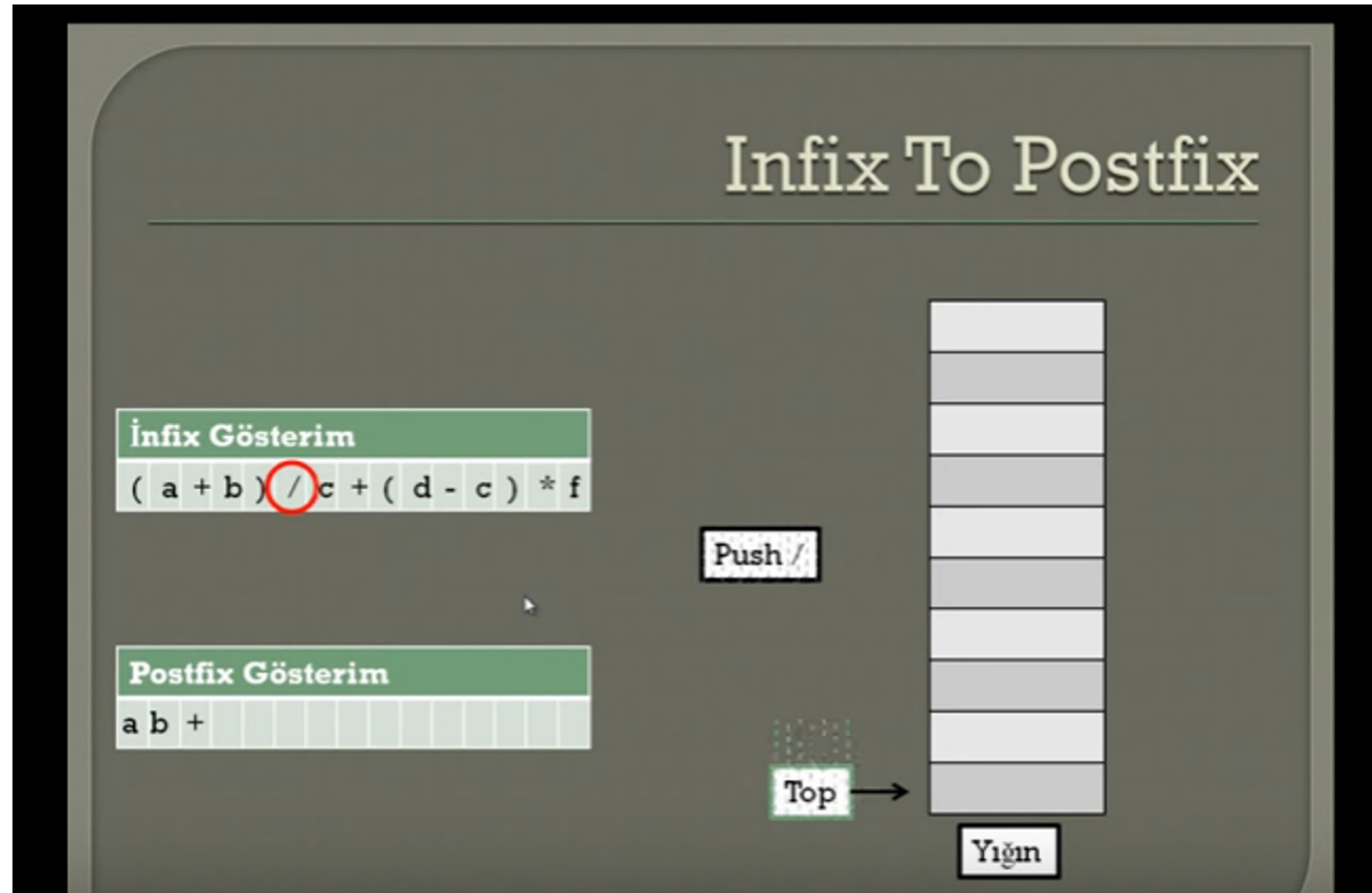


Infix ifadeyi Postfix ifadeye Dönüştürme

- Kapa parantez ile karşılaşınca sıra ile stack'teki operatörleri pop() yapıp post fix'e ekliyoruz taki aç parantezi görene kadar.

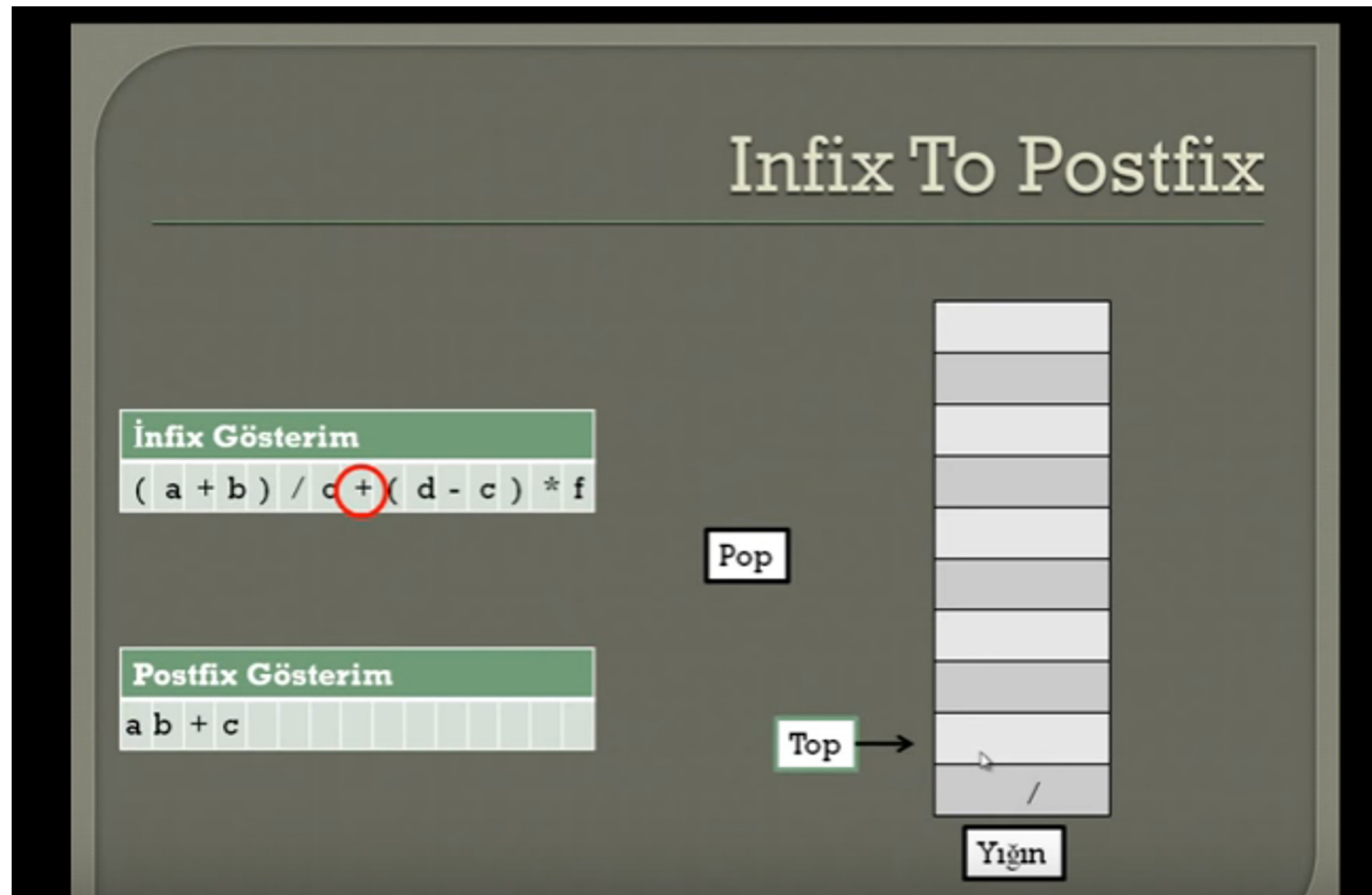


Infix ifadeyi Postfix ifadeye Dönüştürme



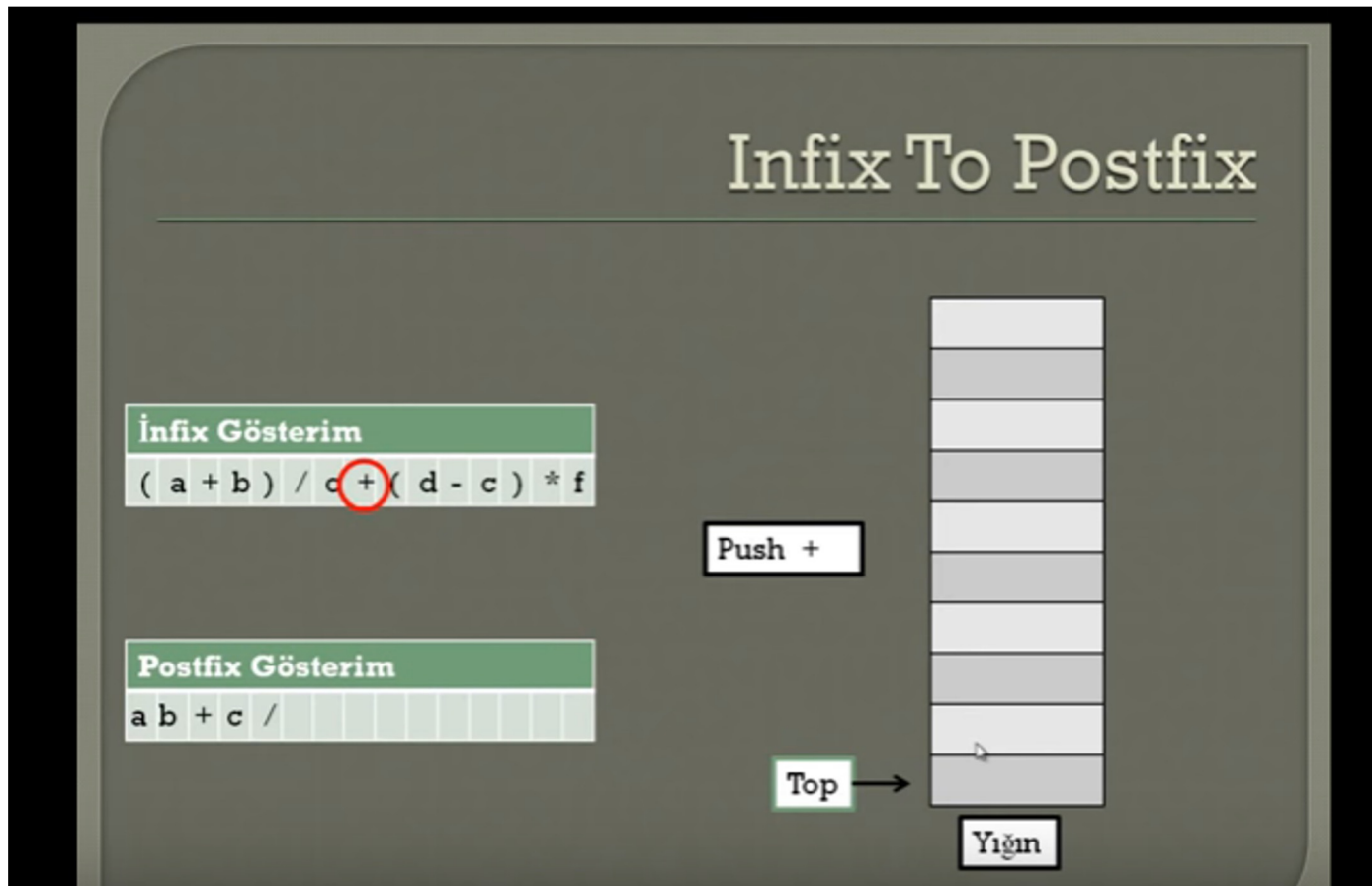
Infix ifadeyi Postfix ifadeye Dönüştürme

- Eğer stack'teki operatör'ün önceliği okunan operatörün önceliğinden büyük veya eşit ise stack'teki operatör pop() edilir.



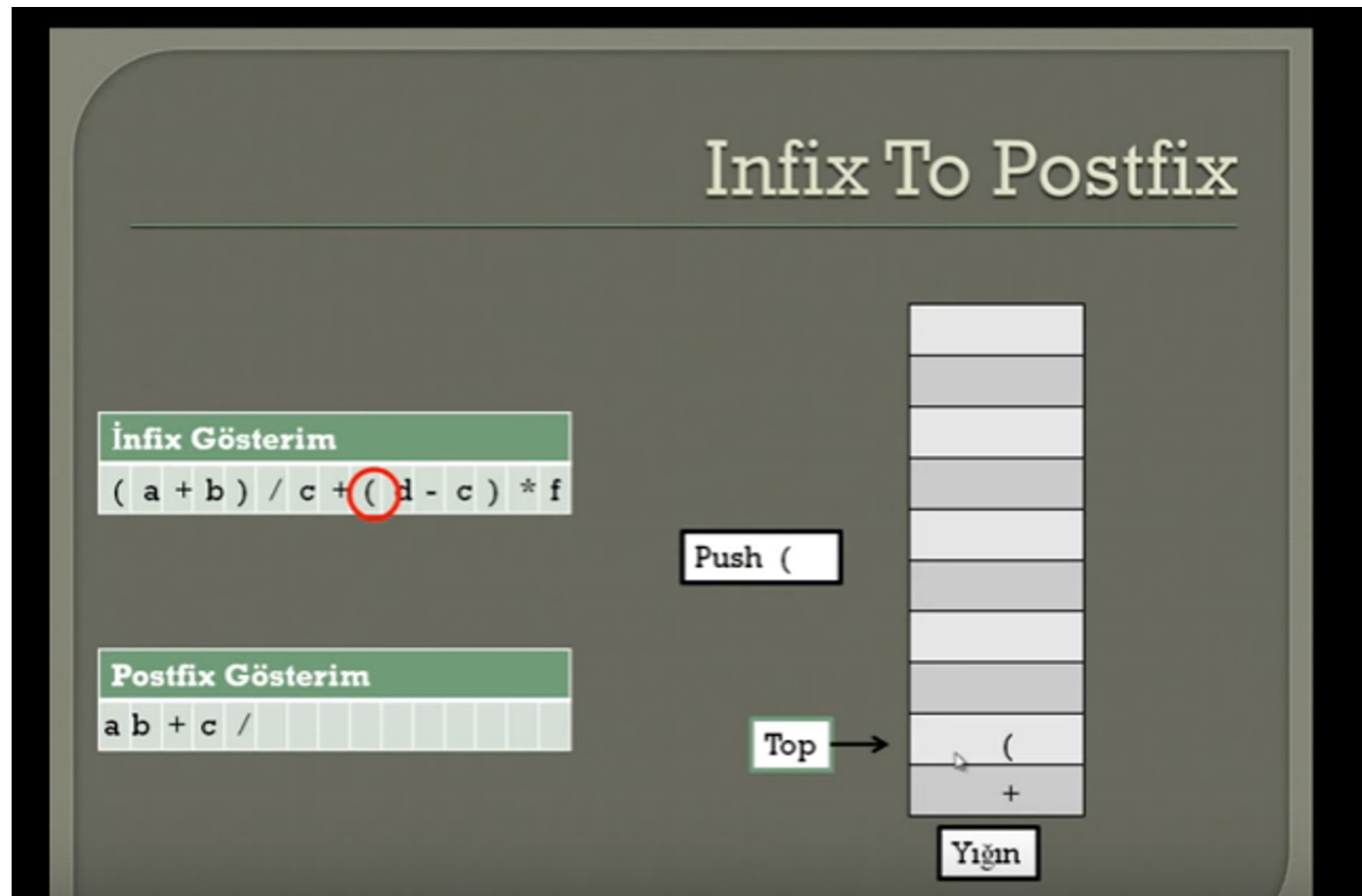
Infix ifadeyi Postfix ifadeye Dönüştürme

- Ve okunan operatör push('+') edilir.

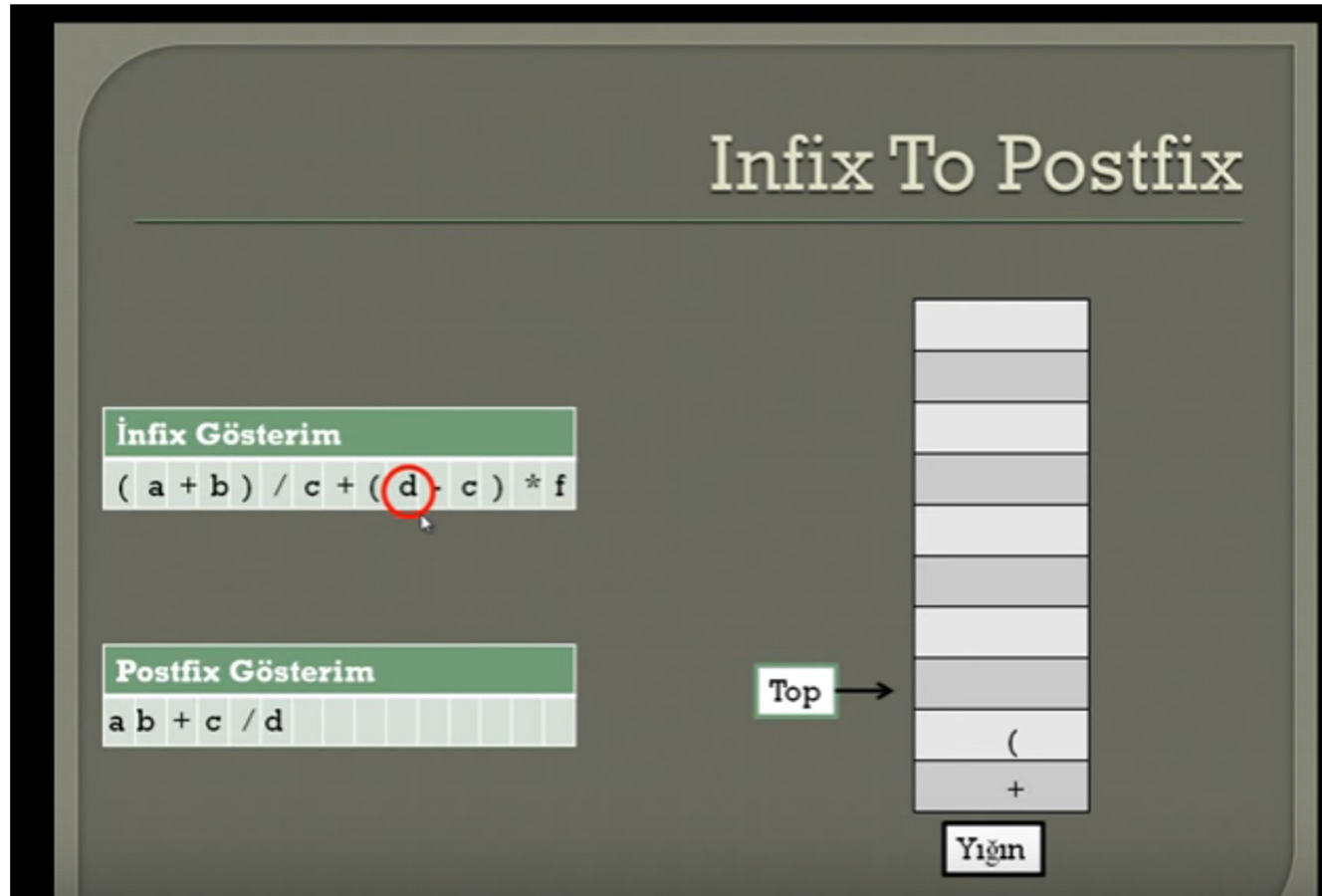


Infix ifadeyi Postfix ifadeye Dönüştürme

- Aç parantez doğrudan stack'e eklenir.

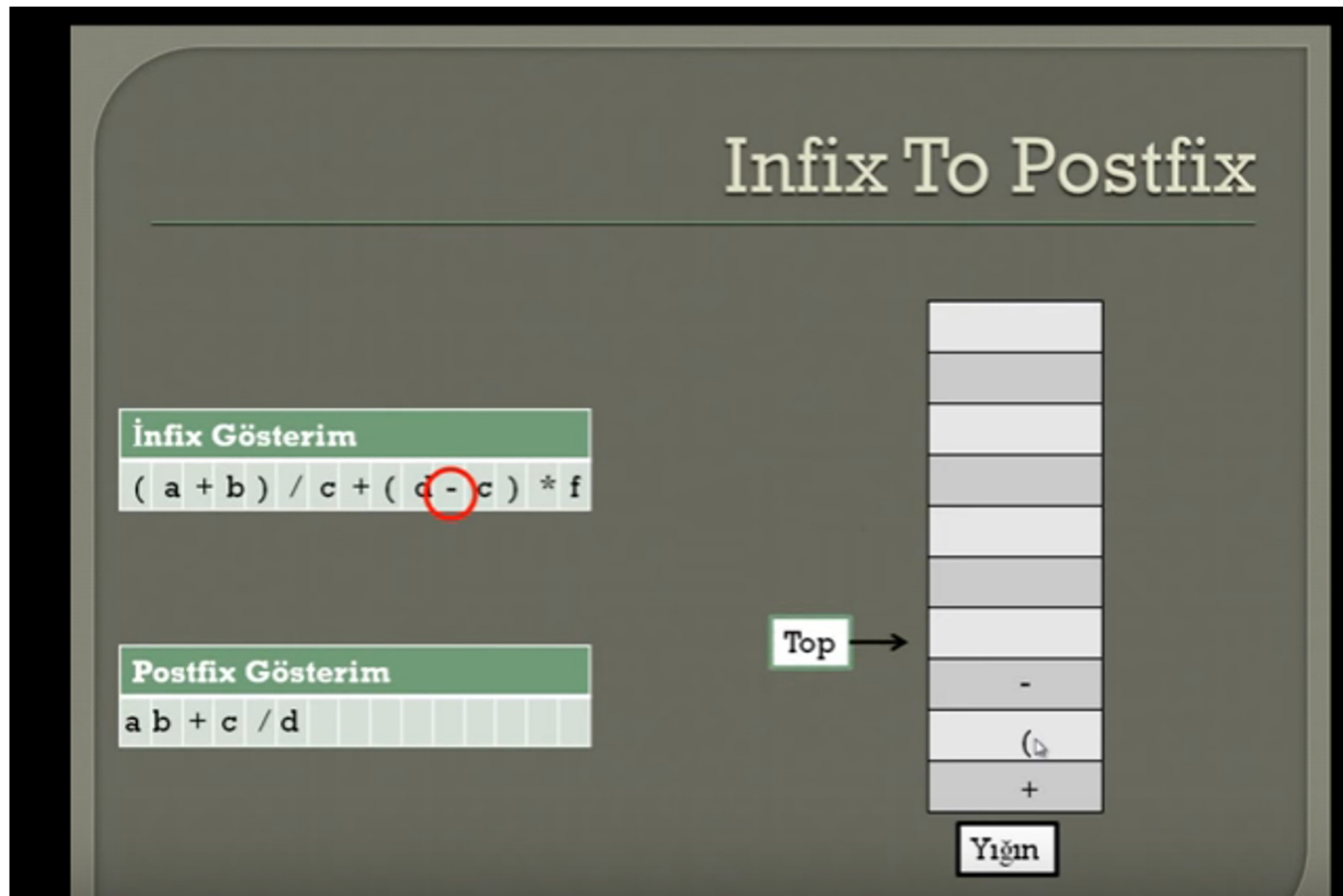


Infix ifadeyi Postfix ifadeye Dönüştürme

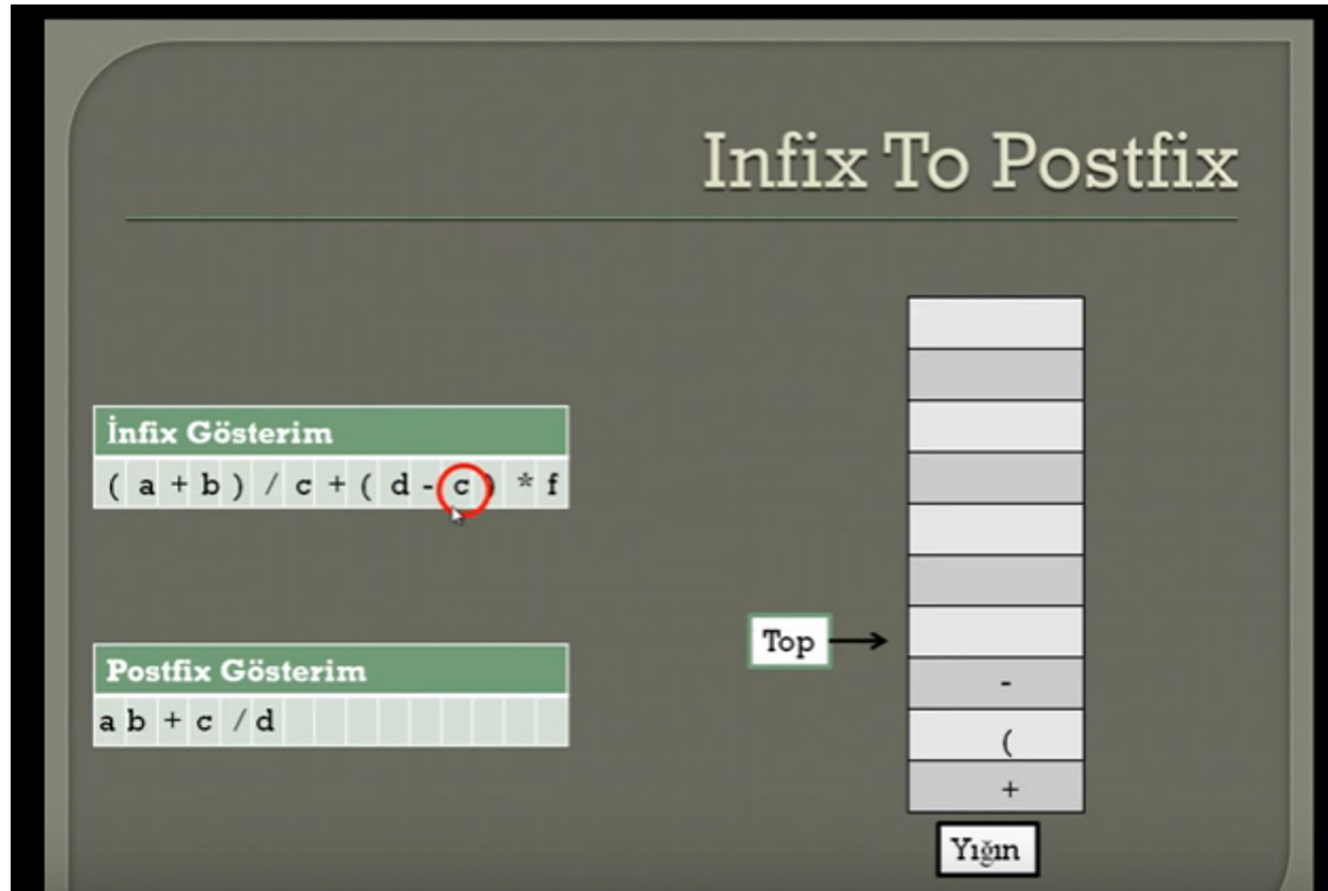


Infix ifadeyi Postfix ifadeye Dönüştürme

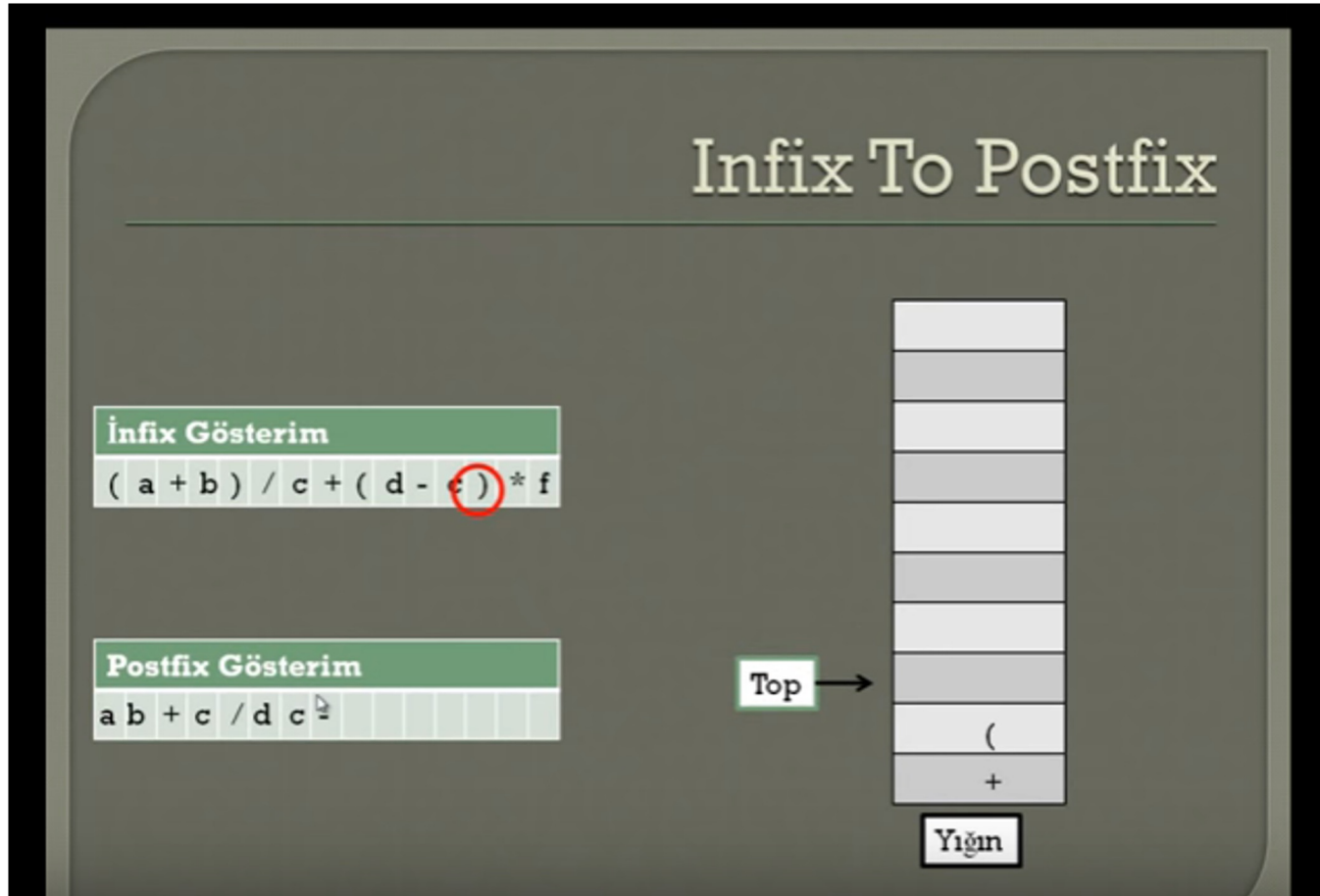
- Aç parantez gördüm okunan operatör push('-') edilir.



Infix ifadeyi Postfix ifadeye Dönüştürme

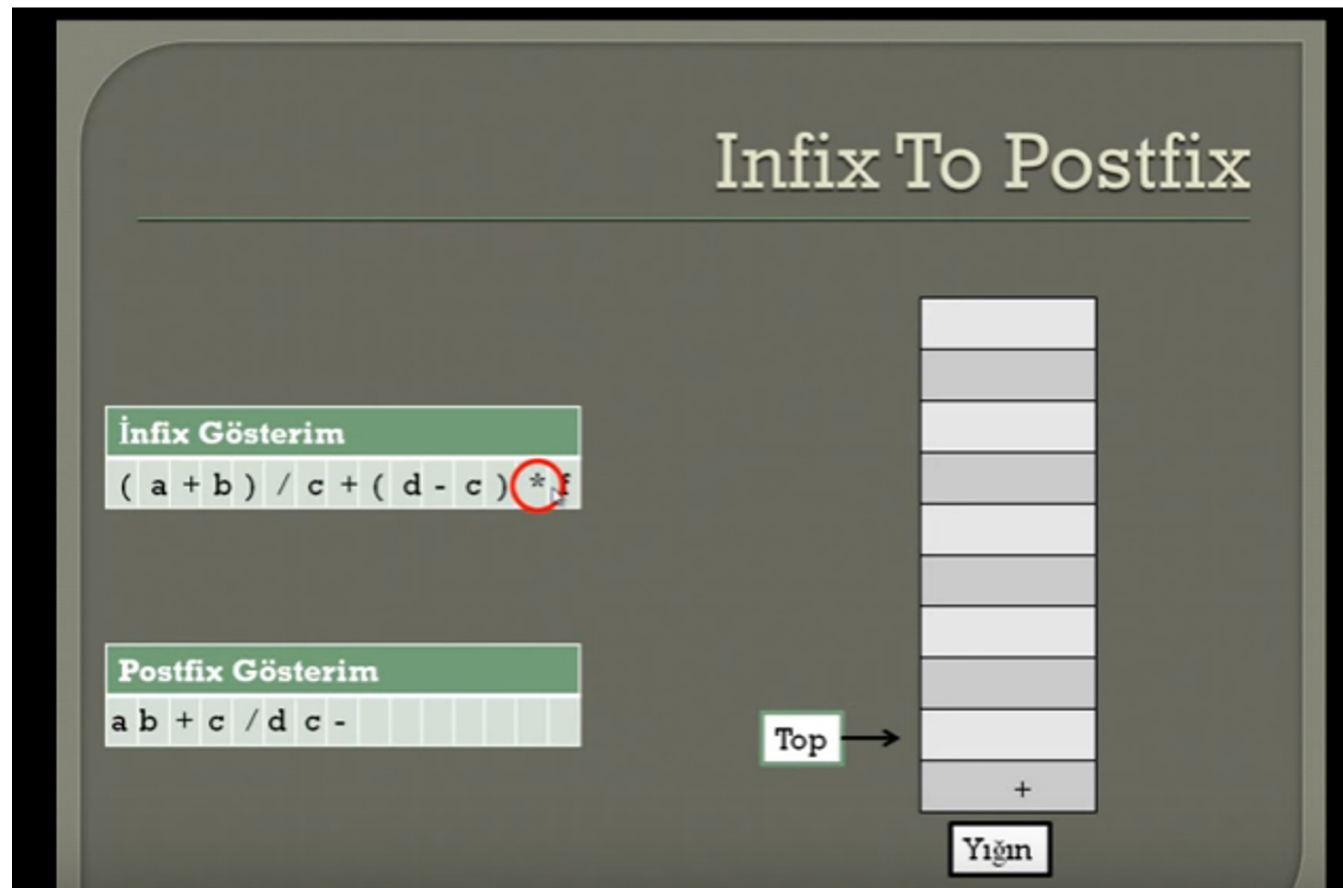


Infix ifadeyi Postfix İfadeye Dönüştürme



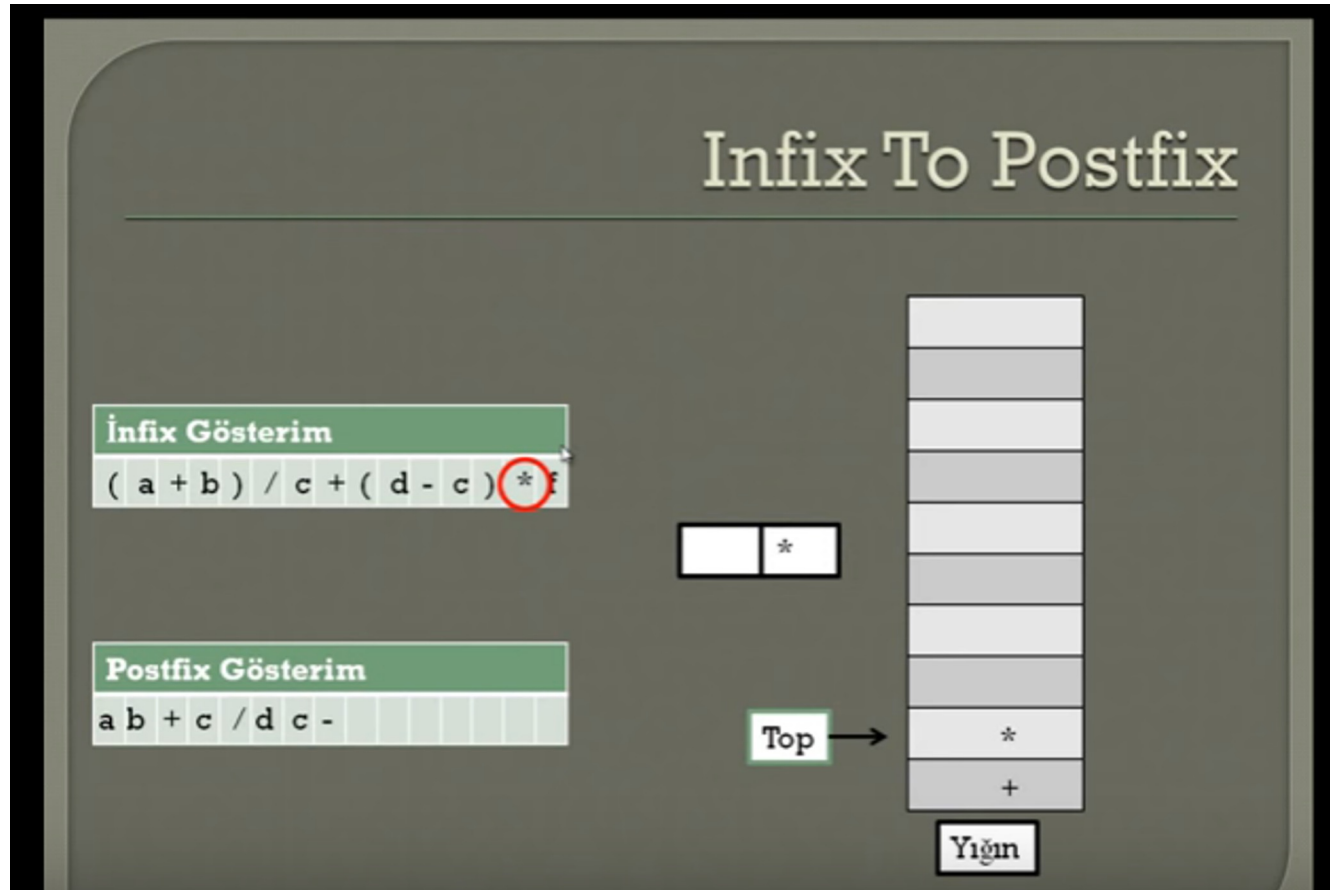
Infix ifadeyi Postfix ifadeye Dönüştürme

- Okunan operatör'ün önceliği stack'te bulunan operatör'ün önceliğinden büyük olduğu için stack'e eklenir.

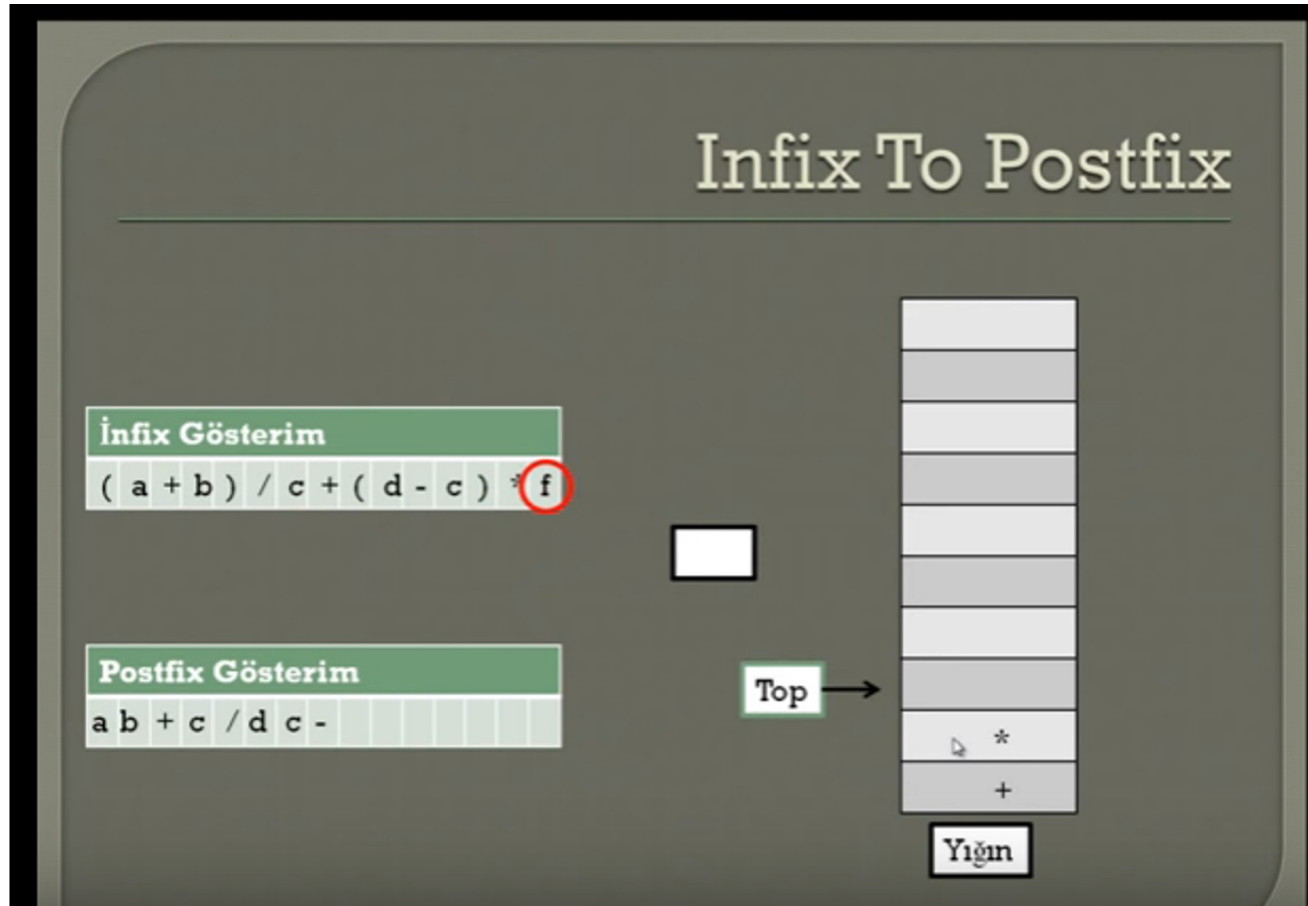


Infix ifadeyi Postfix ifadeye Dönüştürme

- Push('*')

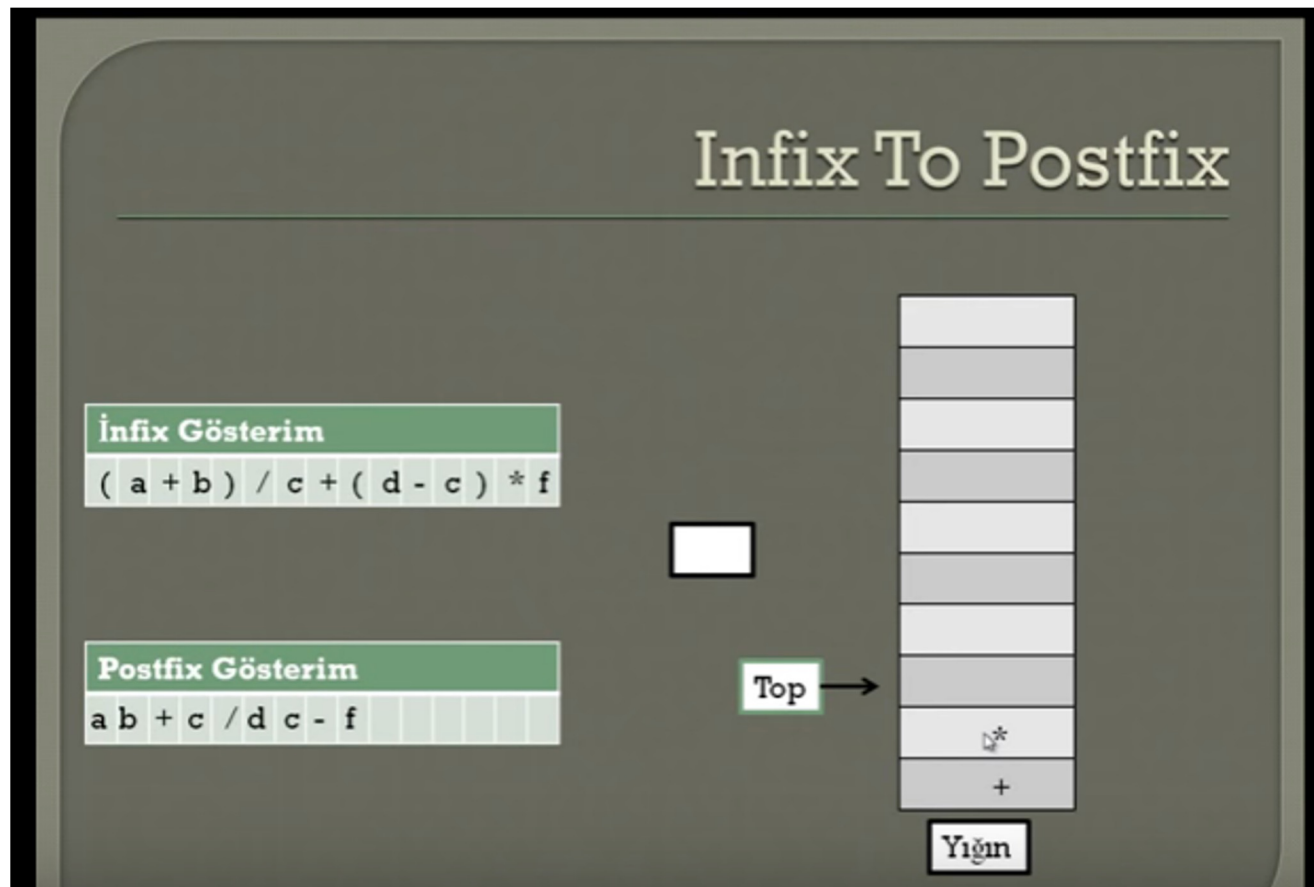


Infix ifadeyi Postfix İfadeye Dönüştürme



Infix ifadeyi Postfix ifadeye Dönüştürme

- Okunacak karakter kalmadı, şimdi ise stack'te bulunan operatörleri post fix'e ekliyoruz.



Infix ifadeyi Postfix ifadeye Dönüştürme

